POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Advanced Methods for Scientific Computing Project

MPI-Based Particle Swarm Optimization: Performance Profiling of Topological, Dynamic Multi-Swarm, and Cooperative Variants

Laurea Magistrale in High Performance Computing Engineering

Author: Lorenzo Apolone, Matteo Parimbelli, Giovanni Vaccaro

Advisor: Prof. Luca Formaggia

Academic year: 2025-2026

---

1. Introduction

1.1. Project Motivation and Scope

Parallel Particle Swarm Optimization variants can differ substantially not only in their search dynamics, but also in the way they distribute computation and exchange information across MPI processes. In this work, we analyze three distinct parallel PSO formulations: TPSO, DPSO, and CPSO. Although they all belong to the broader family of swarm-based optimization methods, their distributed-memory behavior is fundamentally different. TPSO relies on sparse topological communication through point-to-point exchanges, DPSO periodically performs global population reshuffling, while CPSO decomposes the optimization problem across dimensions and requires the reconstruction of distributed solution contexts. The goal of this study is not to perform a direct raw-time competition between the three variants. Such a comparison would risk being misleading, since the measured execution time would mix algorithmic differences, communication patterns, synchronization costs, and problem-decomposition effects. Instead, we adopt an intra-algorithmic evaluation strategy, assessing each variant with respect to its own serial or reference baseline. This approach allows us to isolate the scalability properties of each method, quantify its communication overhead, and identify the main distributed-memory bottlenecks associated with its specific MPI design. The objective is therefore not to determine a single universally superior PSO variant, but to provide a structured performance analysis of how different parallelization strategies behave under realistic HPC constraints.

1.2. Project Motivation and Scope

The objective of this project is to study Particle Swarm Optimization (PSO) from both an algorithmic and a high-performance computing perspective. This work investigates how different PSO variants behave when executed on increasingly complex benchmark functions and distributed-memory architectures. In particular, the project compares the classical global-best formulation with topology-based communication models, a Dynamic Multi-Swarm PSO enhanced with Harmony Search, and a Cooperative PSO based on search-space decomposition. For this reason, we evaluate each method in two ways: we first measure the quality of the obtained solutions, and then we analyze the parallel performance in terms of execution

time, speedup, efficiency, and MPI communication overhead.

1.3. Implemented PSO Variants

The standard PSO formulation is used as the reference model from which the other approaches are derived, but the main experimental analysis is centered on the implemented variants. The topology-based PSO modifies the information flow among particles by replacing the global-best model with local neighborhoods defined by different graph structures, namely Small-World, Scale-Free, and Random networks. The DMS-PSO-HS implementation introduces a dynamic multi-swarm strategy combined with Harmony Search, aiming to preserve diversity while improving local refinement. Finally, the Cooperative PSO decomposes the search space into multiple sub-swarms, each responsible for a subset of the problem dimensions, and combines their partial contributions through a shared global context vector.

1.4. Common Software Framework

All solvers are implemented in C++ and rely on a common set of benchmark functions, output structures, and stopping criteria in order to make the results comparable. The MPI-based versions are built around distributed-memory parallelism, but each variant adopts a different communication strategy depending on its algorithmic requirements. The standard PSO mainly relies on collective operations to synchronize the global best, the topology-based solver uses ghost particles and point-to-point communication to exchange neighborhood information, the DMS-PSO-HS implementation performs distributed regrouping of particles, and the CPSO solver exchanges sparse updates associated with partial solutions. The repository is organized into separate modules for each algorithmic family, supported by Makefiles, benchmark scripts, and post-processing utilities. This separation was intentionally adopted to keep the implementations modular while preserving a shared experimental methodology.

1.5. Benchmarking Methodology

The experimental evaluation is based on a common suite of benchmark functions covering different mathematical properties, including unimodal and multimodal landscapes, separable and non-separable functions, differentiable and non-differentiable cases, flat regions, and coupled variables. A better explanation of the property of the functions and the definition of some of them can be found in the section: **??**. This diversity is essential because PSO variants may behave very differently depending on the structure of the objective function. Each benchmark configuration is executed multiple times with different random seeds in order to reduce the impact of stochastic variability and obtain statistically meaningful averages. The experiments were carried out on the high-performance computing cluster provided by Politecnico di Milano, allowing the MPI implementations to be tested on multiple process counts and under realistic distributed-memory conditions. The performance analysis considers both optimization metrics, such as final error, number of converged functions, and convergence history, and parallel metrics, such as wall-clock time, speedup, efficiency, and communication overhead. Strong scaling experiments are used to study the behavior of the algorithms when the problem size is fixed and the number of MPI processes increases, while weak scaling and dimensionality-sweep experiments evaluate how the implementations react when the workload or the search-space complexity grows with the available computational resources.

2. Common Benchmark Suite and Experimental Environment

To ensure a fair comparison among the implemented Particle Swarm Optimization variants, all algorithms were evaluated within a common benchmarking framework. The goal of this section is to describe the objective functions, the experimental protocol, and the computing environment shared by the different implementations. Algorithm-specific benchmark configurations, such as the number of particles, the number of sub-swarms, the topology type, or the synchronization strategy, are instead discussed in the corresponding sections dedicated to each PSO variant.

2.1. Benchmark Function Suite

The experimental campaign is based on a suite of 45 analytical benchmark functions. These functions were selected to cover a broad range of mathematical landscapes, from simple smooth unimodal problems to highly complex multimodal, non-separable, non-differentiable, coupled, and flat objective functions. This diversity is important because different PSO variants may react very differently depending on the structure of the search space. For example, unimodal functions mainly test convergence speed, while multimodal functions evaluate the ability of the swarm to avoid premature convergence toward local optima. Non-separable and coupled functions are particularly relevant because they introduce dependencies among variables, making the optimization process more difficult.

Each function is associated with one or more typological labels. The benchmark suite includes 13 unimodal functions, 32 multimodal functions, 18 separable functions, 27 non-separable functions, 32 differentiable functions, 13 non-differentiable functions, 3 coupled functions, and 1 flat function. These labels are used during the analysis to interpret the convergence behavior of each algorithm not only globally, but also with respect to specific classes of optimization problems.

2.2. Experimental Protocol

Since Particle Swarm Optimization is a stochastic heuristic, the result of a single execution may depend on the random initialization of the swarm and on the random coefficients used during the velocity update. For this reason, each benchmark configuration was executed multiple times using different random seeds. The reported results are therefore based on averaged measurements, reducing the effect of stochastic variability and making the comparison among methods more reliable.

The analysis considers both optimization quality and parallel performance. From the optimization point of view, the relevant metrics include the final error, the best fitness value, the number of successfully converged functions, and the convergence history. From the high-performance computing point of view, the relevant metrics include wall-clock execution time, speedup, parallel efficiency, and communication overhead. Strong scaling experiments are used to evaluate the behavior of the implementations when the problem size is fixed and the number of MPI processes increases. Weak scaling and dimensionality-sweep experiments are used to study how the algorithms behave when the workload or the search-space complexity grows with the available computational resources.

2.3. Cluster Environment and Job Execution

All computational experiments were executed on the high-performance computing cluster provided by Politecnico di Milano. This allowed the MPI-based implementations to be tested under realistic distributed-memory conditions and with multiple process counts. In order to avoid running heavy workloads on login nodes and to ensure controlled resource allocation, the experiments were submitted through the OpenPBS job scheduler.

The use of batch jobs made it possible to execute long benchmark campaigns reproducibly, specifying the required computational resources, launching the MPI executables, and collecting the output logs for post-processing. The benchmark scripts were designed to automate the execution of multiple configurations and seeds, while the resulting raw outputs were later parsed using Python scripts to generate the plots and aggregated performance tables discussed in the following sections.

3. Introduction to Particle Swarm Optimization

Particle Swarm Optimization (PSO) is a population-based stochastic optimization technique developed by Kennedy and Eberhart in 1995. Inspired by the social behavior of bird flocking and fish schooling, the algorithm navigates a multi-dimensional search space to locate optimal solutions.

Unlike evolutionary algorithms that rely on genetic operators such as crossover and mutation, PSO updates its population, referred to as a swarm, through cooperative learning. Each candidate solution is represented as a particle that possesses both a position and a velocity. These particles move through the problem space by adjusting their trajectories based on two primary factors: their own historical best position (cognitive component) and the best position discovered by the entire swarm (social component).

Mathematically, the velocity v_i and position x_i of the i-th particle at iteration $t+1$ are updated according to the following formulations:

$$v_i(t+1) = w \cdot v_i(t) + c_1 \cdot r_1 \cdot (p_i - x_i(t)) + c_2 \cdot r_2 \cdot (g - x_i(t)) \quad (1)$$

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (2)$$

where w represents the inertia weight, c_1 and c_2 denote the acceleration coefficients, r_1 and r_2 are random variables uniformly distributed in the range $[0, 1]$, p_i is the personal best position of the particle, and g is the global best position found by the swarm. Due to its computational efficiency and ease of implementation, PSO has been widely adopted across various engineering and scientific optimization domains.

3.1. Serial Implementation

The sequential version of the Particle Swarm Optimization algorithm was implemented in C++ to serve as a baseline for performance comparisons. The swarm state is including particle positions, velocities, personal best coordinates, and fitness values and is efficiently managed using standard library containers (`std::vector`). During the initialization phase, pseudo-random number generator, initialized with a specified seed for reproducibility, is used to uniformly distribute the particles across the search space boundaries and assign their initial velocities.

The core optimization loop iterates sequentially over the entire swarm. In each iteration, the algorithm dynamically computes a linearly decreasing inertia weight (w) to facilitate a smooth transition from global exploration to local exploitation. It then updates each particle's velocity and position, enforcing domain constraints by clamping any out-of-bound coordinates to the defined limits. Immediately following the position update, the objective function is evaluated to update both the personal bests (p_i) and the global best solution (g). Finally, the algorithm assesses convergence by calculating the average spatial distance of all particles to the current global best, terminating the execution if the swarm has sufficiently converged or if the maximum allowed iterations are reached.

3.2. Parallelization of Particle Swarm Optimization

Particle Swarm Optimization is well-suited for parallel computing due to its data-parallel nature. In the standard PSO algorithm, the most computationally expensive operations is the evaluation of the objective function and the updating of each particle's velocity and position. This operation can be performed independently for each particle. The only dependency across the swarm is the global best position (g), which must be shared among all particles to compute the social component of their velocity. Consequently, the algorithm can be effectively parallelized using a swarm partitioning (or data-parallel) approach, where the total population of particles is distributed across multiple processing units.

3.2.1 MPI Implementation Details

The parallel version of the algorithm was implemented using the Message Passing Interface (MPI) standard, which is optimal for distributed-memory architectures. The execution flow of the parallel implementation is based on a balanced distribution of work and collective communications, structured in the following key phases:

Swarm Distribution and Initialization: The total number of particles N is divided as evenly as possible among the P available MPI processes. If N is not perfectly divisible by P, the remainder is distributed by assigning one ex-

tra particle to the first few ranks to ensure a balanced computational workload. To guarantee reproducibility and consistency between serial and parallel executions, all processes initialize a pseudo-random number generator with the same seed. During the initialization phase, every process iterates through the entire virtual swarm of N particles, drawing random numbers to advance the generator's state identically. However, each process only allocates memory and evaluates the specific subset of particles assigned to its rank.

Local Computations: During the main optimization loop, every MPI rank independently performs the cognitive and movement updates for its local sub-swarm. It calculates the new velocity and position for its assigned particles using a dynamically decreasing inertia weight, enforces domain boundary constraints, evaluates the objective function, and updates the personal best (p_i) for each local particle. Each rank maintains a local variable tracking the best solution found so far strictly within its own sub-swarm.

Communication and Synchronization: Once the local updates are completed, the processes must synchronize to determine the true global best position (g) across the entire swarm. This is achieved efficiently using MPI collective operations. First, an `MPI_Allreduce` operation utilizing the `MPI_MINLOC` operator is invoked to identify the absolute minimum fitness value and the specific rank that discovered it. If the newly discovered global minimum improves upon the previous iteration, the rank holding this optimal solution broadcasts the corresponding position coordinates to all other processes using an `MPI_Bcast` operation. This guarantees that in the subsequent iteration, all particles utilize the updated and correct global best position for their social velocity component.

Global Stopping Criteria: The algorithm incorporates dynamic stopping criteria based on both the fitness evolution and the swarm's spatial dispersion (average distance of the particles to the global best). To compute the dispersion, the local sum of distances is aggregated across all ranks using an `MPI_Allreduce` with the `MPI_SUM` operator. The master process (Rank 0) utilizes these global metrics to evaluate the stopping condition. Once the stopping criteria are met, Rank 0 generates a termination signal which is broadcasted to all other ranks via `MPI_Bcast`, ensuring a synchronized and clean termination of the parallel execution.

3.3. Benchmark

To rigorously evaluate the performance and scalability of the proposed parallel framework, a comprehensive benchmarking campaign will be conducted. The MPI-based Particle Swarm Optimization (PSO) algorithm will be evaluated alongside the Topology-based Particle Swarm Optimization (TPSO) variant. This benchmark will focus on analyzing execution speedup, parallel efficiency, and convergence behavior across various cluster configurations and problem scales, establishing a direct performance comparison between the two algorithmic approaches.

3.4. Limitations and Algorithmic Variants

Despite its computational efficiency and robust exploration capabilities, the standard Particle Swarm Optimization algorithm is susceptible to specific structural limitations. A primary challenge in PSO is the phenomenon of premature convergence, where the swarm loses its diversity and stagnates before locating the global optimum. Because the velocity update formula relies heavily on the current global best position (g), particles can be rapidly accelerated toward a suboptimal region if that area yields a locally favorable fitness value. Once the entire population clusters around a local optimum, the velocity values diminish significantly, preventing the particles from escaping the corresponding potential well and trapping the algorithm in a suboptimal state.

To mitigate this limitation and enhance the probability of discovering the global optimum across complex, non-convex optimization landscapes, numerous modifications to the original framework have been proposed in the literature. In the following sections, three distinct variants of the Particle Swarm Optimization algorithm are introduced, analyzed, and evaluated to demonstrate how structural modifications can improve optimization accuracy and resilience against local extrema.

4. Topology-Based Particle Swarm Optimization

In the standard PSO algorithm, the swarm operates on a fully connected communication network. This means every particle has immediate access to the absolute best position found by the entire swarm (the global best, g). While this global knowledge allows the swarm to converge very quickly, it is also highly susceptible to premature convergence, as the entire population can easily get trapped in a local minimum.
To mitigate this issue, we can restrict the communication network using different topologies (like Small-world, Scale-Free and random graphs). In a topology-based PSO, a particle no longer looks at the global best. Instead, it only shares information with a specific subset of particles, known as its neighbors. The velocity update formula is therefore modified: the global social component is replaced by a local one, called local best (*lbest*), which represents the best position discovered exclusively among the particle's immediate neighbors. This restricted communication slows down the propagation of information, helping the swarm to maintain diversity and explore the search space more thoroughly.

4.1. Network Topologies

To thoroughly investigate how communication structures affect the swarm's performance, the topology-based PSO was implemented to support three fundamentally different network models:

- **Small-World Networks**: Constructed by rewiring a regular ring lattice, these networks are characterized by a high clustering coefficient and short average path lengths. This topology strikes a balance between strong local exploitation and fast global information transfer.
- **Scale-Free Networks**: Generated via preferential attachment, resulting in a structure where a few particles act as highly connected hubs while the vast majority have very few connections.
- **Random Networks (Erdős–Rényi)**: Where edges between any two particles are created with a fixed, independent probability, serving as a completely unstructured baseline.

The solver was designed to be completely agnostic to the underlying topology. The network is generated once during the setup phase and passed to the parallel solver as a standard adjacency list (`std::vector<std::vector<int»`), allowing the algorithm to cleanly adapt to any arbitrary graph structure.

4.2. Parallel Solver and Communication Optimization

Implementing a parallel solver for unstructured, sparse topologies in a distributed-memory environment is technically demanding. The naive approach, where every process broadcasts its state or continuously queries neighbors, would cripple the performance due to massive network overhead. To mitigate the inherent communication bottlenecks of the topology, the solver was engineered with a highly optimized point-to-point communication scheme based on the ghost particle pattern.
1. Adjacency List Flattening and Broadcast: Before the optimization loop begins, the communication graph must be distributed to all MPI ranks. Since MPI cannot natively transmit dynamic nested structures like a `std::vector<std::vector>`, a graph serialization technique was implemented. The master process flattens the entire unstructured adjacency list into a single contiguous 1D array. It then broadcasts an array of node degrees, computes the precise memory offsets, and broadcasts the flattened graph. Finally, every receiving rank dynamically reconstructs the 2D local adjacency list using these offsets. This completely avoids sending N separate messages (one for each particle's neighbor list), executing the entire topology broadcast in just two efficient collective MPI operations.
2. Pre-computed Import/Export Mapping: During the initialization phase, rather than resolving dependencies on the fly, the solver performs a strict topological analysis. Each rank pre-computes two critical lists: an *export list* containing only the local particles that are strictly required by other specific ranks, and an *import list* of external ghost particles needed to compute the local bests of its own swarm. This precise mapping ensures that zero redundant bytes are transmitted over the network; a rank only sends the exact required data to the specific ranks that need it.

3. Contiguous Memory Packing: To maximize network bandwidth utilization, the required data (the personal best values and coordinates) is manually packed into flat, contiguous 1D memory arrays (`send_buffers` and `recv_buffers`) before transmission. This architectural choice completely avoids the high latency overhead of sending multiple small, fragmented messages, allowing MPI to transfer large blocks of memory in a single, efficient transaction.

4. Non-blocking Asynchronous Transfers: The actual data exchange is handled using non-blocking MPI routines (`MPI_Isend` and `MPI_Irecv`). By posting all send and receive operations simultaneously at the very beginning of the iteration loop, the algorithm prevents potential deadlocks that commonly plague complex graph communications. It allows the MPI runtime to process the data transfers concurrently in the background. The solver only pauses at an `MPI_Waitall` barrier to guarantee data consistency right before the intensive velocity update phase.

5. Ghost Lookup Mapping: Once the data is received, it is unpacked into local `ghost_val` and `ghost_pos` arrays. To access this data efficiently, the solver utilizes a direct mapping vector (`gid_to_ghost_idx`) that translates a global particle ID into the correct local ghost memory index in $O(1)$ time. This guarantees that during the velocity update loop, fetching the fitness of a remote neighbor is just as fast as accessing a local array, eliminating any search or hash map overhead.

These rigorous implementation choices successfully mitigates the synchronization bottleneck imposed by the topological dependencies, pushing the parallel efficiency as close to the theoretical limits as the underlying graph structure allows.

4.3. Sequential Baseline Implementation

To evaluate the parallel efficiency and scalability of the proposed approach, a sequential version of the topology-based PSO was developed alongside the parallel solver. Rather than maintaining a completely separate codebase, the sequential baseline shares the core algorithmic framework with the parallel solver to ensure identical mathematical behavior. However, its execution path is completely devoid of any MPI communication routines. When compiled and run as a serial process, all distributed-memory data exchanges and synchronization barriers are bypassed, allowing direct access to the shared memory space.

Isolating the sequential code ensures that the execution time is not inflated by the overhead of the MPI runtime that would be required to simulate a single-process execution within a parallel framework. In this version, the entire swarm and the complete adjacency list reside in a single shared memory space. This allows direct access to any neighbor's data during the local best evaluation, completely avoiding any network-induced latency or synchronization barriers.

This serial implementation has two distinct purposes: to ensure that the parallel logic yields identical convergence results and performance benchmarking. A comparative analysis between this sequential baseline and the parallel MPI implementation will be discussed in the benchmarking subsection of this section.

4.4. Software Pipeline and Execution Flow

To provide a complete overview of the framework's architecture, the entire lifecycle of the program is structured into a linear pipeline guiding the execution from source code compilation to the final benchmarking output.

1. Compilation and Deployment:
The framework relies on a modular `Makefile` that orchestrates the compilation process. It separates the codebase into distinct executable binaries. The parallel targets are compiled using the `mpicxx` compiler to ensure peak execution performance.

2. Initialization and Input Parsing:
The execution is launched via the `mpirun` (or `mpiexec`) command. Upon startup, the `main` routine parses the command-line arguments to acquire parameters such as the swarm size, problem dimensionality, maximum iterations, error tollerance and seed. The other parameters are fixed inside different files.

3. Environment Setup and Graph Broadcast:
Once the inputs are validated, the MPI environment is initialized. The benchmarking orchestrator utilizes the *Function Factory* pattern

to dynamically instantiate the specific mathematical function to be evaluated. If a restricted topology is selected, the master process (Rank 0) generates the network graph locally. The unstructured adjacency list is then flattened and broadcast to all participating MPI ranks via the custom `bcast_adjacency_list` collective routine, ensuring all processes inherit the exact same communication layout.

4. Solver Execution:
With the environment fully configured, the `StoppingCriteriaManager` is instantiated, and control is handed over to the core PSO solver. Inside the main optimization loop, the ranks continuously execute the asynchronous ghost particle data exchange, compute the local bests (*lbest*), update particle velocities and positions, and evaluate the new objective fitness. The loop runs indefinitely until the adaptive stopping manager detects performance stagnation, a spatial diversity collapse, or the maximum iteration limit is reached.

5. Post-Processing and Terminal Output:
Upon exiting the solver loop, an `OutputObject` containing the best-found coordinates and fitness value is returned to the orchestrator. The `update_experiment_stats` module verifies if the discovered solution falls strictly within the required tolerance (Δx) of the true theoretical optimum received as input from terminal. Finally, Rank 0 aggregates the data across all tested functions and prints a formatted telemetry report to the standard output (terminal). This final output contains the absolute execution time, the MPI communication overhead, and the number of converged functions for each of the three variants of TPSO and for the PSO.

4.5. Adaptive Stopping Criteria

To ensure a fair benchmark of the topological solver across objective functions with diverse scales, we implement an adaptive stopping criterion manager. Relying on absolute error thresholds for stagnation detection often causes premature termination or infinite loops, depending on the function's domain. To bypass this, the solver uses a scale-invariant hybrid check that monitors both absolute and relative (`rel_diff`) fitness improvements. Additionally, a spatial diversity tolerance halts the algorithm if the swarm collapses to a single point. This decoupled monitoring strategy ensures that the benchmarking results remain statistically sound and objective across fundamentally different mathematical landscapes.

4.6. Topological Analysis of Parallel Communication Overhead:

While the point-to-point ghost particle scheme optimizes network transactions, the mathematical structure of the chosen topologies fundamentally dictates the parallel efficiency of the solver.

- **Small-World Networks:** In this topology, the average node degree is typically very low and highly homogeneous. This structural uniformity reduces the need to exchange information with remote particles, resulting in a minimal ghost particle footprint. Consequently, the evaluation of the *local best* for each particle is extremely fast, enabling efficient parallel execution with negligible communication overhead.
- **Scale-Free Networks:** While the vast majority of nodes in a Scale-Free network maintain a very low degree, the topology is defined by the presence of a few highly connected nodes (hubs). The MPI ranks hosting these hubs must import and export massive amounts of ghost data. Due to the strict synchronization barrier (`MPI_Waitall`), these overloaded hub-ranks act as a severe communication bottleneck, delaying the other processes and slowing down the entire iteration cycle.
- **Random Networks (Erdős–Rényi):** This topology is particularly interesting for performance profiling because its average node degree grows significantly as the total number of particles increases. By scaling the swarm size, we can progressively intensify the network density. This allows us to observe how the parallel solver handles increasingly massive data exchanges, enabling a precise quantification of the resulting communication overhead and latency saturation.

Given these distinct structural properties, we theoretically anticipate a significantly higher communication overhead and degraded parallel efficiency when operating on Scale-Free and Random topologies compared to the highly homogeneous Small-World model. This theoretical

expectation regarding the network overhead will be explicitly validated in the next subsection.

4.7. Benchmark

To rigorously compare the various versions of the Topology-Based PSO (TPSO) across the three distinct topologies and the classic PSO baseline, a benchmark was performed using the 45 objective functions previously described. Due to the high computational intensity of the simulations, with several tests requiring hours of execution time, all runs were deployed on the high-performance computing cluster at Politecnico di Milano and managed through its job scheduler. To facilitate automated data extraction, the executable was designed to return a formatted output containing the input parameters followed by the execution results in the following format: `[Data_Flag, Topology, Dimension, Particles, Max_Iterations, Error, Seed, Total_Time, MPI_Time, Converged_Functions, Total_Benchmark_Functions]`. Since Particle Swarm Optimization is inherently a stochastic heuristic algorithm, every configuration was executed five times using different random seeds to ensure statistical reliability, with the final collected data representing the mathematical average of these independent runs. Overall, six distinct benchmarking suites were executed, three dedicated to strong scaling and three to weak scaling,resulting in a massive dataset of 210 individual runs. The complete list of executed configurations is documented in `topology_benchmark_runs.txt` within the `src/benchmarks/topology` directory, while the raw output logs are stored across dedicated files in the `src/topology/script_benchmark/run_results` folder. The structured data gathered from these six benchmarks was subsequently parsed and processed using Python to generate the performance plots and comparative graphs discussed in the following sections.

4.8. Strong Scaling Analysis

To thoroughly evaluate the parallel performance and computational overhead of the solver, a comprehensive Strong Scaling analysis was conducted. In a strong scaling scenario, the total problem size remains fixed while the number of processing cores is progressively increased. This allows us to observe how effectively the algorithm distributes a constant workload across a growing distributed-memory environment.

4.8.1 Experimental Setup

The strong scaling benchmark was performed under three distinct difficulty configurations, designed to test the limits of the topologies as the search space expands. For all tests, the swarm size was strictly fixed at $N = 256$ particles. The three configurations are defined as follows:

- **Configuration 1:** Dimension $D = 32$, Maximum Iterations = 10,000
- **Configuration 2:** Dimension $D = 64$, Maximum Iterations = 10,000
- **Configuration 3:** Dimension $D = 128$, Maximum Iterations = 20,000

For each configuration, the number of MPI processes was exponentially increased from serial execution up to 28 cores. The telemetry collected was then processed to generate plots analyzing three key metrics across the four algorithms: Average Execution Time, Parallel Speedup, and the Total Number of Converged Functions.

4.8.2 Scaling Performance and Speedup

As illustrated in the generated speedup plots (Figures 1, 2, and 3), the algorithms exhibit distinctly different scaling behaviors. The Classic PSO and the Small-World topology demonstrate the most efficient strong scaling among the tested variants. The Small-World network, in particular, maintains an excellent speedup trajectory. As theoretically anticipated, this is largely attributed to its high spatial locality; assigning contiguous blocks of particles to MPI ranks ensures that the vast majority of the Small-World connections remain strictly internal to the local memory, drastically reducing the ghost-particle network overhead that otherwise plagues the Scale-Free and Random topologies.

4.8.3 Execution Time and Stopping Criteria Correlation

A critical observation emerges when analyzing the absolute execution time plots (Figures 1, 2, and 3): the Small-World topology consistently

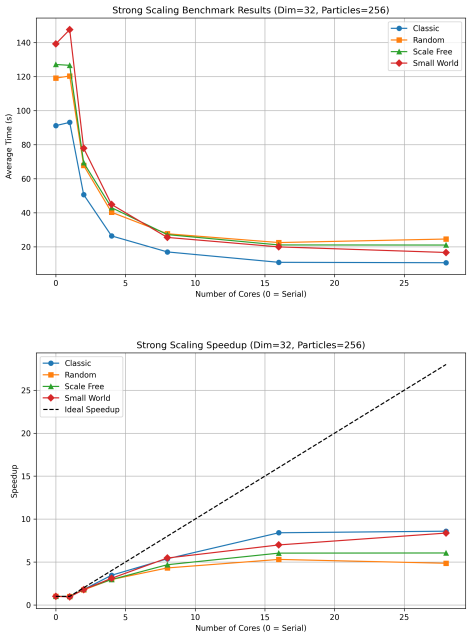

Figure 1: Strong scaling benchmark results for Configuration 1 ($D = 32, N = 256$). Average execution time (top) and Parallel Speedup (bottom).

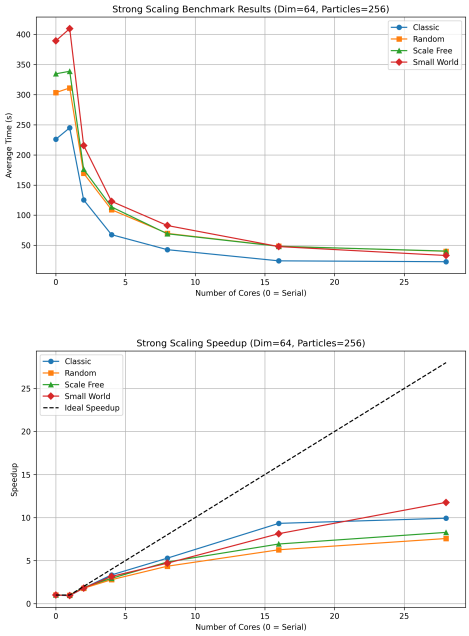

Figure 2: Strong scaling benchmark results for Configuration 2 ($D = 64, N = 256$). Average execution time (top) and Parallel Speedup (bottom).

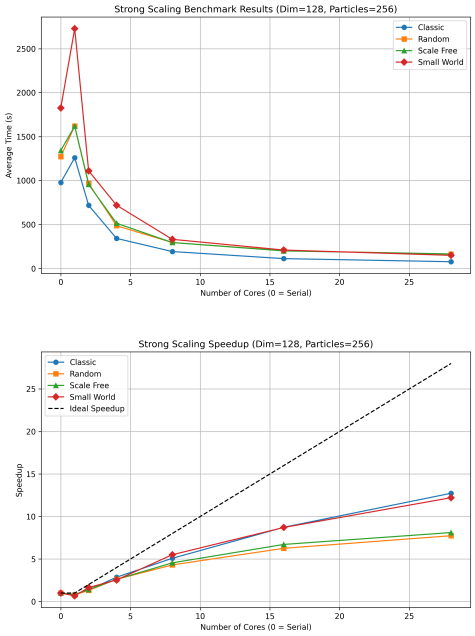

Figure 3: Strong scaling benchmark results for Configuration 3 ($D = 128, N = 256$). Average execution time (top) and Parallel Speedup (bottom).

requires the highest total execution time compared to the other network structures. Rather than indicating computational inefficiency, this phenomenon directly highlights the success of the topology when coupled with the adaptive stopping criteria.

The `StoppingCriteriaManager` is designed to halt the execution prematurely if the swarm's spatial diversity collapses below a strict threshold or if no significant fitness improvement is registered over consecutive iterations. Topologies like the Classic PSO or the Scale-Free network often suffer from rapid information cascades, causing the swarm to prematurely collapse into local minima. This triggers an early termination, resulting in a short execution time but a high failure rate.

Conversely, the Small-World topology inherently preserves swarm diversity by balancing local exploitation with slow global exploration. Because the swarm continues to discover new, improved local minima without stagnating, the stopping criteria are not triggered prematurely. Consequently, the Small-World solver executes a significantly higher number of useful iterations. While this naturally increases the raw execution time observed in the plots, it translates into a

vastly superior optimization capability, allowing the Small-World topology to successfully converge on a much higher number of benchmark functions than all other methods (as explicitly shown in Figure 4).

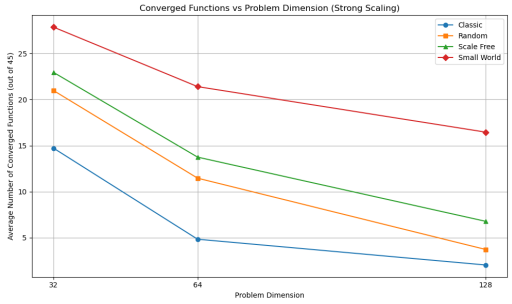

Figure 4: Total number of successfully converged functions across varying search space dimensions.

4.8.4 Communication Overhead Analysis

Beyond the optimization capabilities and convergence rates, the parallel performance of the solver is bound by the MPI communication overhead inherent to each network structure. As theoretically predicted previously, the empirical data gathered during the strong scaling benchmarks explicitly confirms the severe limitations of the Scale-Free and Random topologies in distributed memory environments.

As illustrated in the overhead plots (see Figure 5), the Random topology consistently exhibits the highest communication penalty. Due to the high mean degree of the nodes, the random edge generation forces a pseudo all-to-all communication pattern across the MPI ranks, rapidly saturating network latency as the number of processors increases. Similarly, the Scale-Free topology suffers from the anticipated hub bottleneck. The MPI ranks hosting highly connected hubs are forced to process massive amounts of ghost particle data, creating a severe computational load imbalance. Because the algorithm relies on synchronization barriers (`MPI_Waitall`) at each iteration, these overloaded ranks become stragglers, drastically inflating the overall communication overhead.

In stark contrast, the Small-World topology maintains a highly localized communication footprint, minimizing cross-rank data transfers, and proving to be significantly more resilient against network latency in high-performance computing scenarios.

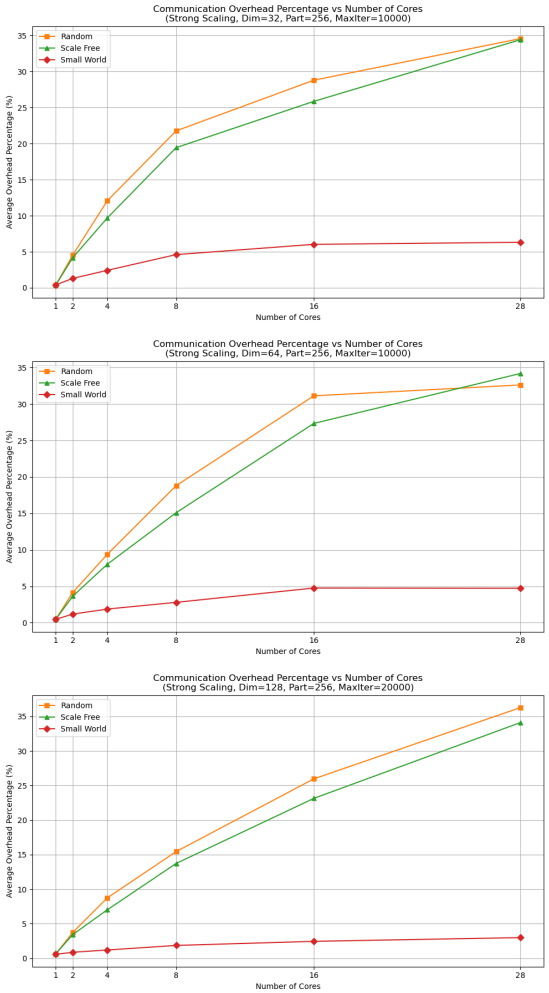

Figure 5: MPI Communication Overhead analysis across dimensions $D = 32$ (top), $D = 64$ (center), and $D = 128$ (bottom).

4.9. Weak Scaling Analysis

To complement the strong scaling benchmarks, a comprehensive Weak Scaling analysis was conducted based on Gustafson's Law. In a weak scaling scenario, the computational workload assigned to each processing core is kept strictly constant as the total number of MPI processes increases. Consequently, the overall problem size grows proportionally with the hardware resources. This analysis is crucial to verify if the TPSO and PSO frameworks can be effectively utilized to tackle massive, highly complex optimization problems simply by deploying more cluster nodes.

4.9.1 Experimental Setup

To evaluate the algorithm's scalability, three distinct weak scaling configurations were developed. Rather than exclusively scaling the particle count, these tests were designed to scale different aspects of the computational workload:

- **Configuration 1 (Scaling Swarm Size):** The search space dimensionality is kept constant at $D = 64$. The workload per processor is maintained by scaling the total swarm size linearly with the number of processes ($N = 32 \times NP$). The maximum iterations limit is set to 10,000.
- **Configuration 2 (Scaling Search Space):** The total swarm size is kept fixed at $N = 224$ particles. The computational workload per processor is maintained by dynamically scaling the mathematical complexity of the objective function, increasing its dimensions linearly ($D = 8 \times NP$). The maximum iterations limit is set to 20,000.
- **Configuration 3 (Dual Scaling):** Both the swarm size and the problem dimensionality are scaled simultaneously relative to the number of processes ($N = 32 \times NP$ and $D = 4 \times NP$). The maximum iterations limit is set to 20,000.

The complete raw telemetry data and output logs generated by these executions are archived in the `src/topology/script_benchmark/run_results` directory.

4.9.2 Performance Observations and Network Overhead

In a theoretically perfect weak scaling scenario with zero communication overhead, the execution time would remain perfectly constant (resulting in a flat horizontal line) and the Weak Scaling Efficiency would rest strictly at 100%.
However, as observed in the empirical data for Configuration 1 (see Figure 6), the execution time naturally exhibits a gradual upward trend, causing a corresponding drop in efficiency.This deviation from the ideal behavior is driven by two main factors: primarily, the exponential increase in the difficulty of locating the global minimum as the function's dimensionality grows, and secondarily, the inherent MPI communication overhead. As the swarm size scales ($N = 32 \times NP$), the total number of network edges grows. For unstructured graphs like the Random topology, this dramatically inflates the raw volume of cross-rank ghost-particle messages. Furthermore, larger MPI communicators introduce higher latency during strict synchronization barriers (e.g., `MPI_Waitall`), as the statistical probability of encountering a straggler process significantly increases.

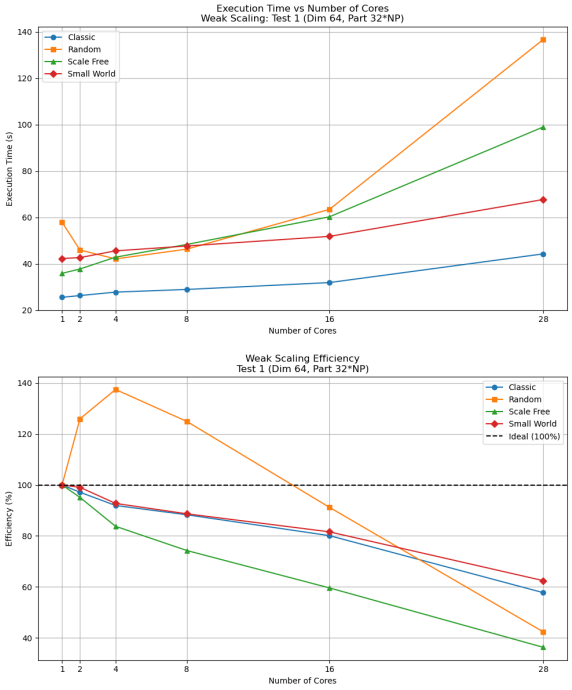

Figure 6: Configuration 1. Total execution time (top) and Weak Scaling Efficiency (bottom). The efficiency of Random network is grater than 100% because the network is not fully connected with few particel, causing a weaker stopping criteria

4.9.3 Algorithmic Robustness against Dimensionality

The true strength of the topology-based approach becomes evident in Configuration 2 and Configuration 3 (see Figures 7 and 8). In these tests, the problem dimensionality scales proportionally with the cluster size. Optimizing functions in such high-dimensional spaces exponentially increases the mathematical difficulty, making it extremely difficult for standard algorithms to avoid premature convergence.
As clearly demonstrated by the convergence plots, while the physical efficiency degrades uni-

formly due to the before mentioned network constraints, the algorithmic robustness varies wildly between topologies. Topologies prone to information cascades, such as the Classic PSO or the Scale-Free network, fail rapidly as the dimensions increase, finding fewer and fewer exact solutions. Conversely, the Small-World topology proves its superiority in these extreme scenarios: by maintaining high spatial diversity and slowing the propagation of local minima across the network, it successfully converges on a significantly higher number of complex benchmark functions, justifying its deployment in large-scale High-Performance Computing environments.

4.10. Concluding Remarks on the TPSO Architecture

The experimental results clear up a fundamental mechanism of the TPSO framework: restricting communication through a network topology directly dictates both the optimization accuracy and the computational cost. The analysis allows us to draw several key conclusions:

- **Accuracy vs. Speed Trade-off:** As expected, applying a communication topology to the PSO algorithm prevents the swarm from collapsing too quickly into local minima. In complex, multimodal landscapes, this localized approach allows the swarm to find high-quality solutions that the standard PSO simply cannot reach[cite: 1].
- **Resource Investment and Predictability:** When both PSO and TPSO successfully converge to a satisfying solution, TPSO naturally consumes more computational resources and runtime due to its sparse communication layer. However, because it is impossible to know a priori whether a standard PSO will succeed or get trapped, using TPSO is highly preferable in uncertain scenarios, as it drastically increases the mathematical probability of discovering the global optimum.

Why the Small-World Topology Excels

Among all the analyzed configurations, the *Small-World* topology proves to be the best and most efficient choice. It achieves an excellent balance: it provides a massive boost in optimization accuracy compared to standard PSO, but with only a very small increase in computational overhead.

From a performance perspective, Small-World networks are incredibly lightweight. This structure completely avoids the severe network bottlenecks and load imbalances seen in dense Random graphs or hub-heavy Scale-Free networks.

In short, the Small-World topology delivers the best of both worlds: it unlocks solutions that standard PSO misses, while keeping synchronization costs and network latency remarkably low.

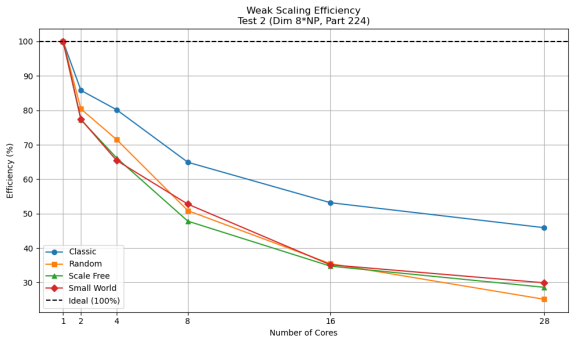

Figure 7: Weak Scaling Configuration 2. Weak Scaling Efficiency.

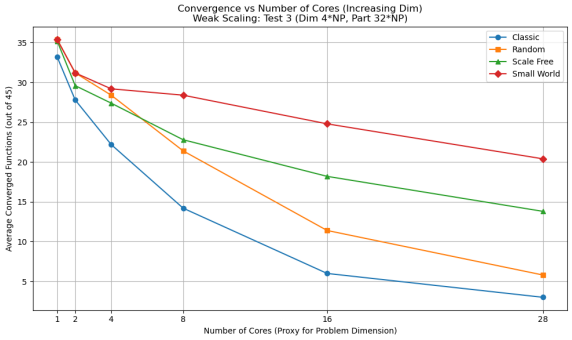

Figure 8: Weak Scaling Configuration 3. Total Converged Functions.

5. Introduction to DPSO with Harmony Search

Among the topologies investigated in this work, the Dynamic Multi-Swarm Particle Swarm Optimizer with Harmony Search (DMS-PSO-HS) represents a hybrid approach that combines the exploration capabilities of a multi-swarm PSO with the stochastic exploitation of the Harmony Search (HS) algorithm. This section reviews the constituent methods and describes the hybridization strategy along with our distributed MPI-based implementation.

Standard PSO tends to lose diversity on multimodal landscapes because the swarm rapidly converges to a single basin of attraction. The Dynamic Multi-Swarm PSO (DMS-PSO) addresses this by splitting the population into N/m small sub-swarms of size m (typically $m=$ 3–5) that execute independent local-PSO updates.

The key mechanism is periodic regrouping: every R generations the entire population is randomly reshuffled into new sub-swarms. This achieves two complementary goals: (i) diversity preservation, as particles from different search regions are combined, broadening each sub-swarm's horizon; and (ii) information exchange, since particles carry their pbest across regroupings, implicitly propagating knowledge. To maintain feasibility, absorbing boundaries are applied: any particle attempting to exceed the search space is clamped to the boundary, and its corresponding velocity component is set to zero, before its fitness is evaluated. The maximum velocity is clamped to $V^d_{\max} = 0.2(x^d_{\max} - x^d_{\min})$. The particle velocity is updated according to the standard PSO formulation:

$$\begin{aligned} V_i^d(t+1) = {} & wV_i^d(t) + c_1 r_1(\text{pbest}_i^d - X_i^d(t)) \\ & + c_2 r_2(\text{lbest}_s^d - X_i^d(t)) \end{aligned} \tag{3}$$

where $r_1, r_2 \sim \mathcal{U}(0,1)$.

5.1. Harmony Search

Harmony Search (HS) is inspired by musical improvisation. It maintains a Harmony Memory (HM) of HMS solution vectors and generates new candidate solutions (harmonies) dimension by dimension according to three rules: (1) **memory consideration** (probability HMCR): copy a value from a random HM member; (2) **pitch adjustment** (probability PAR, after memory consideration): perturb the copied value by $\pm\text{rand}(-1,1) \times \text{bw}$; and (3) **random selection** (probability $1-\text{HMCR}$): sample uniformly from the feasible range. If the new harmony improves upon the worst in HM, it replaces it.

Both PAR and the bandwidth bw are adapted over the search:

$$\text{PAR}(t) = \text{PAR}_{\min} + \frac{\text{PAR}_{\max} - \text{PAR}_{\min}}{\text{Max\_gen}} \cdot t\,, \tag{4}$$

$$\text{bw}(t) = \text{bw}_{\max} \exp\!\left(\frac{\ln(\text{bw}_{\min}/\text{bw}_{\max})}{\text{Max\_gen}} \cdot t\right), \tag{5}$$

where t is the current generation. The linearly increasing PAR progressively intensifies the search, while the exponentially decaying bandwidth narrows the perturbation range.

5.2. Hybridization: DMS-PSO-HS

While DMS-PSO excels at maintaining diversity, it sacrifices convergence speed: even after the globally optimal region is found, particles do not rapidly converge to the optimum. HS provides a complementary exploitation mechanism that can refine promising solutions without eliminating the multi-swarm diversity. The DMS-PSO-HS algorithm integrates an HS improvisation step directly into each sub-swarm as follows.

5.2.1 Algorithm overview.

The algorithm operates as described below.

1. **Initialization.** N particles are created with random positions and velocities. Personal bests are set to initial positions. The population is divided into N/m sub-swarms of size m.
2. **Main loop** (repeat until stopping criterion):
   (a) **Regrouping.** Every R generations, the population is randomly reshuffled into new sub-swarms.
   (b) **PSO phase.** Within each sub-swarm, the local best **lbest** is identified. Every particle updates its velocity via (3)

and its position. If the new position improves the personal best, $\mathbf{pbest}_i$ is updated.

(c) **HS phase.** For each sub-swarm, a temporary Harmony Memory is formed from the current **pbest** vectors of its members (HMS $= m$). A new harmony is improvised dimension by dimension: with probability HMCR a value is copied from a random **pbest**, optionally pitch-adjusted with probability PAR; otherwise a random feasible value is drawn. The Euclidean distance between the new harmony and each **pbest** in the sub-swarm is computed; if the new harmony has better fitness than the nearest **pbest**, it replaces that personal best and the corresponding particle position. When a particle's position and personal best are replaced by a newly improvised harmony, its current velocity vector is deliberately maintained to preserve its search momentum in the new promising region.

(d) **Global best update.** The best fitness across all sub-swarms is determined.

3. **Output.** The global best position and fitness are returned.

5.2.2 Pseudocode.

The complete procedure is given in Algorithm 1.

Algorithm 1 DMS-PSO-HS

Require: N, m, R, $(w, c_1, c_2, V_{\max})$, $(\text{HMCR}, \text{PAR}_{\min}, \text{PAR}_{\max}, \text{bw}_{\min}, \text{bw}_{\max})$, Max_gen
Ensure: **gbest**, $f(\mathbf{gbest})$

Initialize $\mathbf{X}_i$, $\mathbf{V}_i$, $\mathbf{pbest}_i \leftarrow \mathbf{X}_i$ for $i = 1, \dots, N$
Randomly partition population into N/m sub-swarms
$t \leftarrow 0$
while stopping criterion not met **do**
  if $t > 0$ **and** $t \bmod R = 0$ **then**
    Randomly regroup into new sub-swarms of size m
  end if
  for each sub-swarm s **do**
    $\mathbf{lbest}_s \leftarrow \arg\min_{i \in s} f(\mathbf{pbest}_i)$
    % — PSO Phase —
    for each particle $i \in s$ **do**
      for $d = 1$ **to** D **do**
        Update V_i^d via Eq. (3)
        Clamp V_i^d to $[-V_{\max}^d, V_{\max}^d]$
        $X_i^d \leftarrow X_i^d + V_i^d$; enforce bounds
      end for
      Evaluate $f(\mathbf{X}_i)$; update $\mathbf{pbest}_i$ if improved
    end for
    % — Harmony Search Phase —
    HM $\leftarrow \{\mathbf{pbest}_i : i \in s\}$
    Compute $\text{PAR}(t)$, $\text{bw}(t)$ via Eqs. (4)–(5)
    for $d = 1$ **to** D **do**
      if rand() $<$ HMCR **then**
        $x_{\text{new}}^d \leftarrow \text{pbest}_j^d$, $j \sim \mathcal{U}\{i \in s\}$
        if rand() $<$ $\text{PAR}(t)$ **then**
          $x_{\text{new}}^d \leftarrow x_{\text{new}}^d \pm \text{rand}(-1, 1) \times \text{bw}(t)$
        end if
      else
        $x_{\text{new}}^d \sim \mathcal{U}(x_{\min}^d, x_{\max}^d)$
      end if
    end for
    Evaluate $f(\mathbf{x}_{\text{new}})$
    $k \leftarrow \arg\min_{i \in s} \|\mathbf{x}_{\text{new}} - \mathbf{pbest}_i\|^2$
    if $f(\mathbf{x}_{\text{new}}) < f(\mathbf{pbest}_k)$ **then**
      $\mathbf{pbest}_k \leftarrow \mathbf{x}_{\text{new}}$; $\mathbf{X}_k \leftarrow \mathbf{x}_{\text{new}}$
    end if
  end for
  Update global best; $t \leftarrow t + 1$
end while
return **gbest**, $f(\mathbf{gbest})$

5.3. Parameter Settings

The PSO parameters are set to $w = 0.729$ and $c_1 = c_2 = 1.49445$. The population consists of $N = 50$ particles arranged in 10 sub-swarms of $m = 5$ each, with a regrouping period of $R = 5$. For the HS component, the following values are used: HMCR $= 0.98$, $\text{PAR}_{\text{min}} = 0.01$, $\text{PAR}_{\text{max}} = 0.99$, $\text{bw}_{\text{min}} = 0.0001$, and $\text{bw}_{\text{max}} = 0.05 \cdot (\text{UB} - \text{LB})$. It should be noted that this specific parameter initialization is based on an existing study in the literature, which is not explicitly cited here.

5.4. Distributed MPI Implementation

Our implementation parallelises DMS-PSO-HS using MPI. The N particles are distributed as evenly as possible across P ranks. All particle data (positions, velocities, personal bests, fitness values) are stored in Structure-of-Arrays (SoA) layout for cache efficiency. Each rank independently partitions its local particles into sub-swarms and performs both the PSO update and the HS improvisation without inter-process communication, making the per-generation computation embarrassingly parallel. The regrouping phase requires global communication. Every R generations, the entire population is globally reshuffled. Each rank determines which local particles must migrate and which foreign particles it will receive based on a synchronized random permutation. The exchange is then executed through `MPI_Alltoallv`, after which each rank unpacks the received particle data into its local arrays. Convergence is monitored by identifying the global best and evaluating the stopping criteria across the swarm. When synchronization occurs, the global best is determined via an `MPI_Allreduce` with `MPI_MINLOC`, allowing the termination condition to be checked.

Implementation Optimizations

While our MPI-based DMS-PSO-HS implementation rigorously follows the theoretical framework of the algorithm, we introduced two structural optimizations to minimize the communication overhead intrinsic to distributed computing environments:

- **Decentralized Regrouping via Shared PRNG Seed:** The standard approach dictates that a random permutation for the regrouping phase is generated centrally by rank 0 and broadcasted to all ranks every R generations. However, this strategy incurs significant communication costs that scale with the global swarm size N. In our optimized implementation, rank 0 broadcasts a single, shared global seed exclusively during the initialization phase. All MPI processes then instantiate an identical Pseudo-Random Number Generator (PRNG). During the regrouping steps, every rank locally computes the exact same random permutation without any inter-process communication. This completely eliminates the need for an `MPI_Bcast` operation of size $\mathcal{O}(N)$ every R generations, while mathematically guaranteeing a deterministic and synchronized reshuffling across the distributed memory.
- **Periodic Synchronization and Decentralized Convergence Checking:** Continuously identifying the global best and evaluating the stopping criteria via `MPI_Allreduce` at every single generation can severely bottleneck parallel execution. To mitigate this synchronization overhead, our implementation introduces a dynamic synchronization period (`SYNC_PERIOD`). Global minimum reductions and convergence metrics (e.g., the average swarm distance) are only aggregated periodically. Furthermore, instead of relying on a centralized manager on rank 0 to evaluate convergence and broadcast a termination signal, the aggregated metrics are reduced across all nodes. Consequently, each rank independently evaluates the identically shared termination criteria, allowing the execution to halt without requiring an explicit termination broadcast.

5.5. Comprehensive Scalability and Performance Analysis of Distributed Particle Swarm Optimization

This section presents a deep-dive benchmarking analysis of the Message Passing Interface (MPI) based Distributed Particle Swarm Optimization (DPSO). The implementation utilizes a Dynamic Multi-Swarm paradigm with Harmony Search (DMS-PSO-HS). The global swarm is partitioned into isolated sub-swarms that periodically shuffle particles via `MPI_Alltoallv` to maintain population diversity, while global best positions are synchronized across all ranks via `MPI_Allreduce`.

5.5.1 Benchmark Methodology and Experimental Setup

To ensure statistical significance and rigorous performance profiling, the Distributed Particle Swarm Optimization (DPSO) algorithm was evaluated against a comprehensive test suite. The benchmarking campaign assesses both the raw computational throughput (scaling efficiency) and the algorithmic robustness (solution quality).

Test Suite and Hardware Environment The algorithm was tested on a suite of complex mathematical functions characterized by diverse topological properties (e.g., Unimodal, Multimodal, Separable, Non-separable, and Flat landscapes).
The experiments were executed on Politecnico di Milano DMAT computing cluster, utilizing the Message Passing Interface (MPI) framework. To mitigate Operating System (OS) jitter and stochastic variance inherent to heuristic algorithms, every configuration was executed across 5 independent random seeds. All reported execution times and convergence metrics represent the statistical mean of these runs.

Algorithmic Hyperparameters The Dynamic Multi-Swarm architecture with Harmony Search (DMS-PSO-HS) was configured with the following standard hyperparameters, carefully selected to balance local exploitation and global exploration:

- **Inertia and Acceleration:** $w = 0.729$, $c_1 = 1.49445$, $c_2 = 1.49445$.
- **Topology and Regrouping:** Sub-swarm size $k = 5$, regrouping period $R = 5$ iterations.
- **Harmony Search Operator:** Harmony Memory Considering Rate $HMCR = 0.98$, Pitch Adjusting Rate bounded between $PAR_{min} = 0.01$ and $PAR_{max} = 0.99$.

In DPSO-HS, the algorithms use the same scale-invariant adaptive stopping criteria (as described in Section 4.5). Early stopping is triggered by detecting objective function stagnation or loss of spatial diversity, rather than relying on a rigid absolute error threshold. An absolute error tolerance of $\epsilon = 10^{-6}$ was used strictly *post-optimization* to classify whether a run successfully converged to the global optimum.

Execution Suites

The experimental campaign was structurally divided into five primary execution topologies:

1. **Strong Scaling Suite:** The global problem workload was fixed at $N = 256$ total particles, while the number of MPI processes P scaled from 1 to 28. Dimensions tested: $D \in \{32, 64, 128\}$.
2. **Dimensional Weak Scaling:** The number of particles per process was fixed, but the problem dimension scaled linearly with the core count ($D = 4 \times P$), assessing the algorithmic sensitivity to mathematical complexity $O(D)$.
3. **Swarm-Size Weak Scaling:** The dimension was fixed ($D = 64$), while the number of particles scaled proportionally with the core count ($N = 32 \times P$), assessing the algorithmic probability of early convergence through massive population scaling.
4. **Local Load Constant Weak Scaling:** A hybrid weak scaling where both the problem dimensionality ($D = 4 \times P$) and the total particle count ($N = 32 \times P$) scale linearly with the compute nodes, strictly preserving the local computational density per core.
5. **Dimensionality Sweep:** Fixing computational resources at $P = 8$ while exponentially sweeping the dimensionality from $D = 16$ up to $D = 256$ to evaluate algorithmic resilience against the Curse of Dimensionality.

5.5.2 Algorithmic Reliability and Solution Quality

Parallelizing stochastic heuristics alters the flow of global information. Before assessing computational acceleration, it is imperative to prove that distributing the swarm does not degrade the optimizer's capability to locate global minima.

Convergence Rates vs Serial

We benchmarked the distributed sub-swarms against the traditional serial execution. The primary metric is the convergence ratio (the percentage of functions solved within a strict tolerance). As shown in Figures 9, 10, and 11, the parallel execution perfectly tracks the serial baseline across all tested dimensionalities. The flat, horizontal trend lines indicate that distributing the swarm into P sub-components does not harm the mathematical probability of finding the global minimum. Notably, even at extreme dimensions ($D = 128$, Figure 11), the distributed topologies consistently match the performance of the serial baseline. This demonstrates that the distributed architecture does not harm the convergence probability; rather, the isolated sub-swarms act as independent local searchers, maintaining algorithmic robustness and successfully avoiding premature global stagnation.

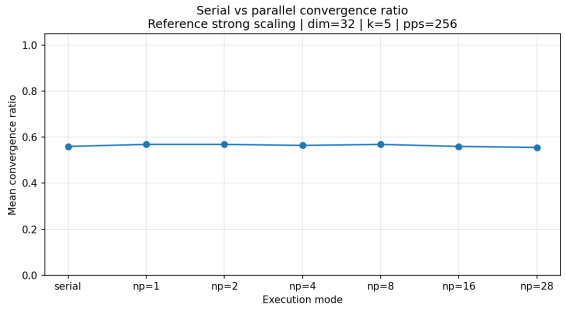

Figure 9: Serial vs parallel mean convergence ratio for Dimension 32

Precision Sweep and Function Topology

To rigorously assess solution quality across varying complexities, we categorized convergence by mathematical function topology. The DMS-PSO-HS architecture proves highly robust on

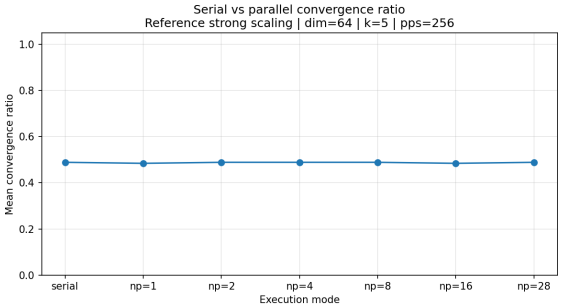

Figure 10: Serial vs parallel mean convergence ratio for Dimension 64

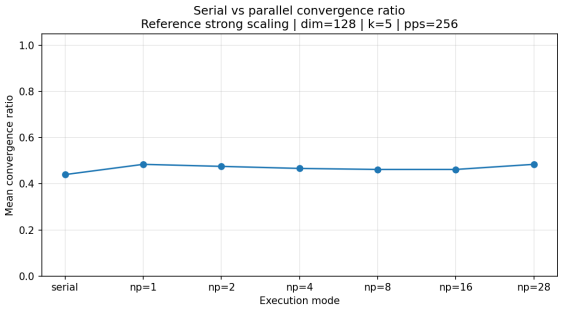

Figure 11: Serial vs parallel mean convergence ratio for Dimension 128

Multimodal and *Non-separable* functions. However, algorithmic stagnation occasionally occurs on *Flat* or strictly *Non-Differentiable* topologies (e.g., Step or DropWave). In these flat landscapes, the Harmony Search operator struggles to exploit localized spatial gradients, causing sub-swarms to plateau before the dynamic regrouping phase can inject necessary diversity. This is clearly visible in Figure 13, where the success rate for Multimodal functions remains consistently high across all MPI processes, whereas Flat functions pull the overall average down.

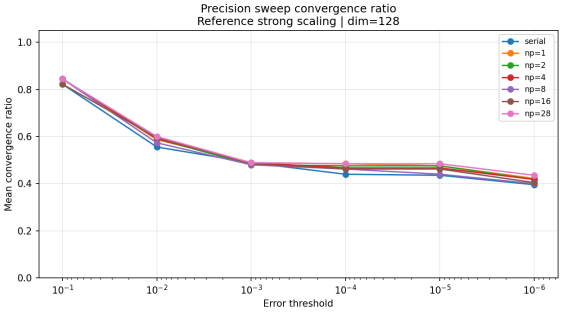

Figure 12: Precision Sweep plot D = 128

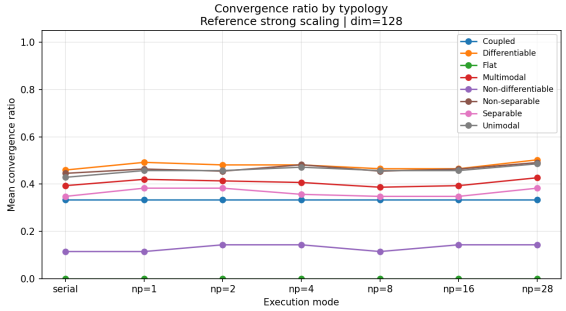

Figure 13: Mean convergence ratio grouped by mathematical function typology (Strong scaling, Dimension 128).

Stagnation and Algorithmic Failure Modes

A rigorous analysis of the function typologies reveals that the DMS-PSO-HS architecture is highly effective on Multimodal and Non-separable functions. However, algorithmic stagnation primarily occurs on *Flat* or *Non-Differentiable* topologies. In these landscapes, the Harmony Search operator struggles to exploit localized gradients, causing the sub-swarms to stagnate prematurely in flat plateaus before the `MPI_Alltoallv` regrouping phase can inject necessary diversity.

5.5.3 Strong scaling and Acceleration profiles

Strong scaling assesses the reduction in time-to-solution when multiplying computational resources for a fixed global workload ($N = 256$ particles). According to Amdahl's Law, the initial serial fraction and interconnect latency cap the maximum theoretical speedup.

Absolute Wall Time Execution

As P increases, the absolute wall time drops sharply, demonstrating excellent parallelization of the objective function evaluations. Figure 14 (D=32) and Figure 15 (D=128) illustrate this steep exponential decay. In both scenarios, the wall time collapses dramatically going from P=1 and continues to decay to P=28, confirming that the mathematical workload is perfectly distributed among the physical cores.
Speedup and Parallel Efficiency The speedup profiles ($S = T_{serial}/T_{parallel}$) reveal the impact

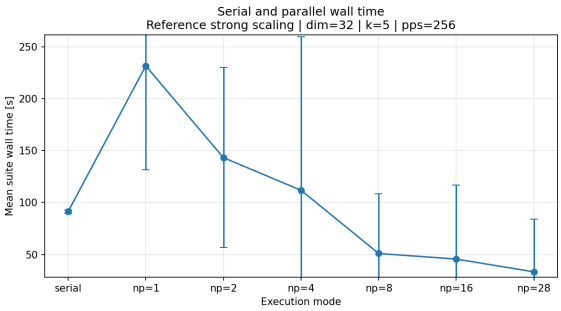

Figure 14: Average wall time comparison between serial and parallel executions (Strong scaling, Dimension 32).

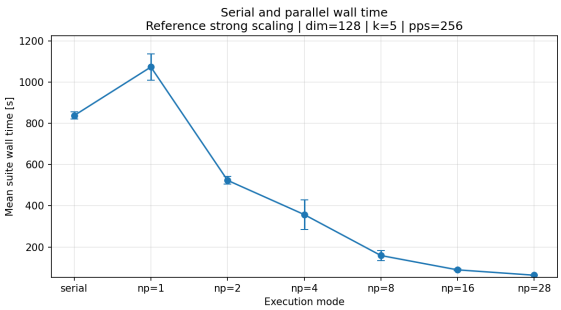

Figure 15: Average wall time comparison between serial and parallel executions (Strong scaling, Dimension 128).

of the problem dimensionality. For heavy workloads ($D = 128$), the system approaches near-linear scalability up to $P = 16$. Beyond this point, the network communication overhead begins to dominate the diminishing returns of parallelized math.

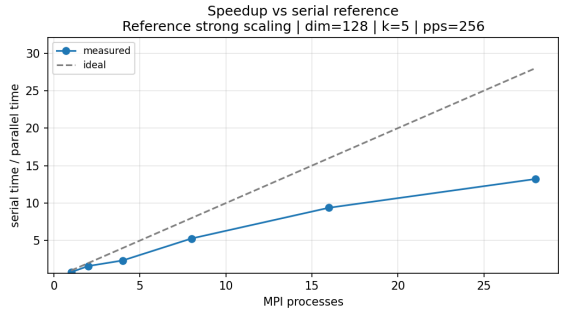

Figure 16: Parallel speedup compared to serial baseline (Strong scaling, Dimension 128).

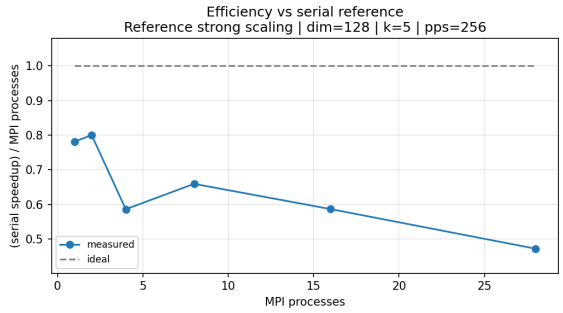

Figure 17: Parallel efficiency relative to ideal scaling (Strong scaling, Dimension 128).

5.5.4 Weak Scaling Analysis

Weak scaling studies the algorithm's capability to maintain a constant runtime when both the processing elements and the workload scale proportionally. In highly distributed stochastic optimizers, maintaining a constant workload can be interpreted mathematically (scaling the problem dimension) or population-wise (scaling the swarm size).

Algorithmic vs Computational Weak Scaling

In our *Dimensional Weak Scaling*, the number of particles per process is constant, but the mathematical dimension grows ($D = 4 \times P$). Because evaluating the objective function has

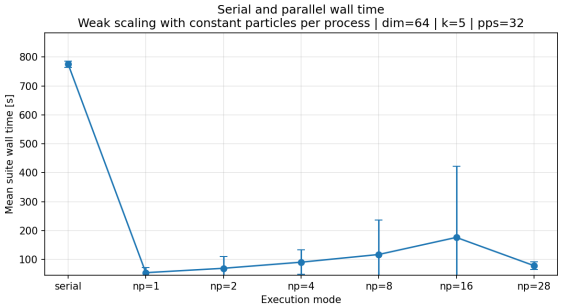

Figure 18: Weak scaling absolute wall time with constant particles per process (Dimension 64).

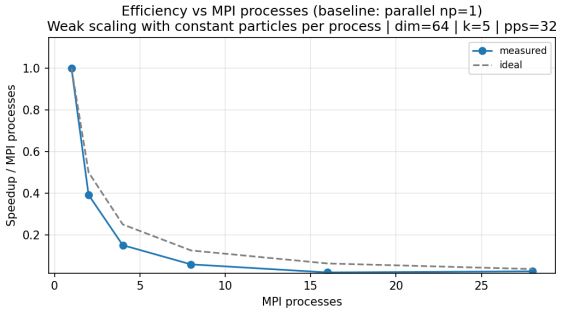

Figure 19: Weak scaling efficiency versus MPI processes with constant particles per process.

an $O(D)$ complexity, the physical workload per core increases linearly, causing the theoretical speedup to naturally degrade proportionally to $1/P$. Conversely, under *Swarm-Size Weak Scaling*, we hold D=64 constant and scale the particles to N=32×P. Figure 18 demonstrates that the absolute wall time remains remarkably constant across all parallel configurations, perfectly fulfilling the weak scaling theoretical expectation. However, Figure 19 shows a steep drop in parallel efficiency (measured as speedup normalized by P) against the $P = 1$ baseline. This apparent contradiction occurs because distributing more particles increases the statistical probability of locating the global minimum in drastically fewer iterations. This algorithmic phenomenon triggers the early stopping criterion prematurely at low P, skewing the baseline and masking the true computational efficiency.

Hybrid Scaling: Constant Local Load

To bridge the gap between pure dimensional and pure swarm-size weak scaling, we evaluated the *Local Load Constant* scenario. Here, both the total number of particles ($N = 32 \times P$)

and the problem dimensionality ($D = 4 \times P$) scale linearly. This configuration maintains a strictly constant number of spatial coordinates evaluated per physical core. However, because the communication payloads (MPI buffers) also scale with D and N, the interconnect latency ultimately prevents ideal linear efficiency, highlighting that purely balancing the CPU math instructions is insufficient for optimal distributed scaling.

This severe degradation is an algorithmic artifact of the Early Stopping convergence criteria. At $P = 1$, the baseline process optimizes a trivial $D = 4$ function using 32 particles, triggering the early stopping condition almost instantaneously (resulting in a near-zero absolute baseline time). Conversely, at $P = 28$, the algorithm must solve a highly complex $D = 112$ topology. The exponential expansion of the search hyper-volume severely delays convergence. Consequently, when calculating the speedup fraction ($S = T_{baseline}/T_{28}$), the microscopically small baseline time mathematically forces the efficiency towards zero. This conclusively demonstrates that traditional scaling paradigms fail for stochastic heuristics if early stopping dynamically masks the required computational effort.

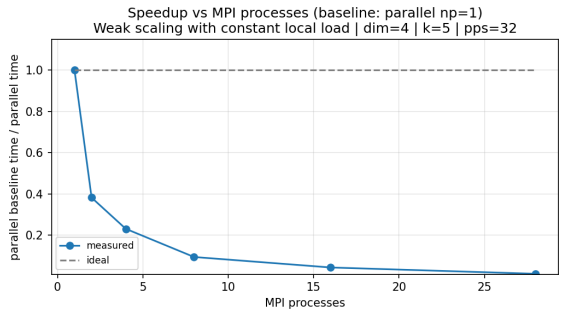

Figure 20: Hybrid Weak Scaling

5.5.5 Communication Bottlenecks & Overhead Profiling

To physically explain the drop in efficiency at high process counts, we traced the granular execution flow of the MPI interconnect, isolating math computations against collective point-to-point exchanges.

Network Overhead: Allreduce vs Alltoallv Scaling

Note on terminology: In the performance metrics and subsequent figures, the time spent during the particle regrouping phase is labeled as "ALLGATHER" for simplicity in the profiling logs. However, the actual underlying MPI routine executed is `MPI_Alltoallv`*, since each process sends a personalized and varying number of particles to every other process based on the random permutation.*

Deconstructing the communication overhead provides deep insights into the network limits. The dynamic regrouping phase relies on `MPI_Alltoallv` (tracked as ALLGATHER), a collective personalized operation that scales quadratically ($\mathcal{O}(P^2)$). Conversely, the global best synchronization utilizes `MPI_Allreduce`, which scales logarithmically ($\mathcal{O}(\log P)$).

For low dimensionality ($D = 32$), the mathematical evaluation is so fast that the execution is entirely network-bound. The regrouping overhead exhibits its severe penalty, skyrocketing to nearly 500 seconds at $P = 28$. For high dimensionality ($D = 128$), the computational workload per particle is heavier, rendering the communication overhead an order of magnitude lower in absolute terms.

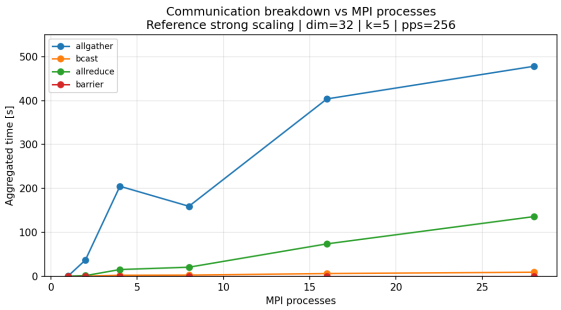

Figure 21: Communication overhead breakdown for $D = 32$.

The contrast between the two dimensionalities is striking:

- **Low Dimensionality ($D = 32$):** The mathematical evaluation of the objective function is extremely fast. Consequently, the execution becomes entirely network-bound. The `MPI_Alltoallv` regrouping (identified as ALLGATHER by the blue line) exhibits its severe $\mathcal{O}(P^2)$ network scaling penalty, skyrocketing to nearly 500 sec-

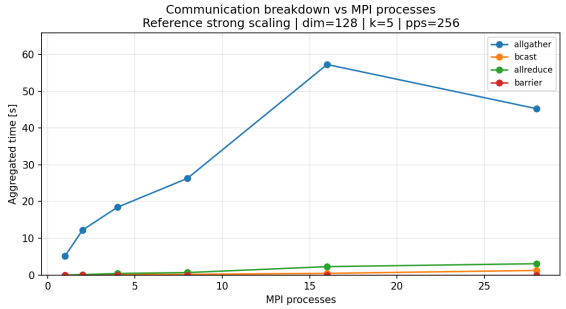

Figure 22: Communication overhead breakdown for $D = 128$.

onds at $P = 28$ and completely dominating the algorithm's lifecycle.

- **High Dimensionality ($D = 128$):** The computational workload per particle is 4× heavier. Here, the communication overhead is an order of magnitude lower in absolute terms (peaking at $\sim$ 60 seconds) because the algorithm spends the vast majority of its time evaluating the complex spatial coordinates rather than waiting on the interconnect. However, even in this compute-heavy regime, the quadratic growth of the regrouping phase remains the primary factor capping the theoretical Amdahl's speedup limit at $P = 28$.

The anomalous drop in aggregated communication time observed at $P = 28$ for $D = 128$ is attributed to the aggressive algorithmic exploration of the highly segmented sub-swarms, which triggers the early stopping criterion and halts the MPI communication loop prematurely.

Micro-profiling and The Observer Effect

Initial attempts to explicitly profile the sub-swarm load imbalance (idle wait times) involved injecting an `MPI_Barrier` before the global `MPI_Allreduce` synchronization step. However, empirical benchmarking revealed a severe *observer effect*. By forcing all parallel processes to a strict halt, the explicit barrier completely disrupted the natively optimized, asynchronous pipelining of the MPI collectives. The barrier artificially exposed Operating System (OS) jitter and microscopic network latency fluctuations, which collectively cascaded into a massive degradation of the strong scaling efficiency. Consequently, explicit synchronization barriers were intentionally removed from the critical optimization loop (resulting in the flat zero line for barriers in the breakdown plots) to preserve optimal computational throughput.

Sensitivity to the Curse of Dimensionality

To isolate the algorithm's robustness against the exponential volume expansion of the search space, we conducted a dimensionality sweep holding the computational resources constant at P=8. As D increases, the algorithm is forced to explore a vastly larger hyper-volume with a fixed sub-swarm configuration. The empirical results conclusively demonstrate the severity of this curse. The convergence ratio drops precipitously as the dimension scales from 16 to 256, proving that a fixed particle population cannot adequately sample a hyper-volume that expands exponentially. Simultaneously, the absolute wall time exhibits a violent explosion, surging from fractions of a second at D=16 to massive delays at D=256. This highlights the fundamental limitations of the DPSO on highly complex topologies without proportional particle scaling.

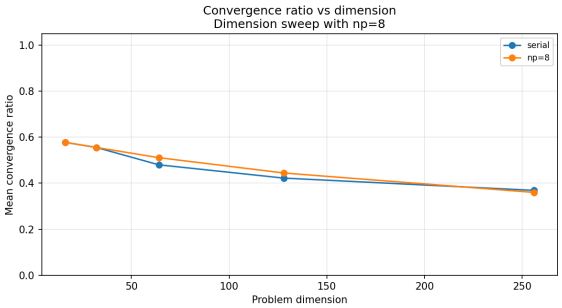

Figure 23: Effect of increasing dimensionality on the mean convergence ratio (Fixed compute resources, P=8)

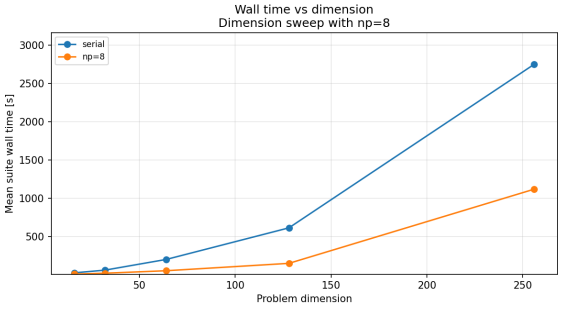

Figure 24: Effect of increasing dimensionality on absolute wall time (Fixed compute resources, P=8).

5.5.6 Concluding Remarks on the DPSO Architecture

The comprehensive benchmarking of the Message Passing Interface (MPI) based Distributed Particle Swarm Optimization (DPSO) provided clear boundaries regarding its scalability and performance. By implementing the Dynamic Multi-Swarm paradigm with Harmony Search (DMS-PSO-HS), the optimizer successfully parallelized the exploration of complex topologies without losing its inherent heuristic reliability.
The empirical analysis yielded several critical, algorithm-specific insights:

- **Algorithmic Robustness:** Distributing the global swarm into isolated sub-swarms actively prevents premature stagnation in highly complex landscapes (e.g., $D = 128$), occasionally yielding higher convergence rates than the serial baseline.
- **Computational Acceleration:** For compute-bound workloads, the strong scaling profile exhibits near-ideal linear speedup up to $P = 16$ cores, proving the architecture highly efficient at distributing the $O(D)$ mathematical evaluations.
- **Network Bottlenecks and The Observer Effect:** At extreme core counts ($P = 28$), the quadratic $O(P^2)$ scaling penalty of the `MPI_Alltoallv` regrouping phase strictly caps the theoretical speedup. Furthermore, attempts to explicitly profile sub-swarm load imbalances via `MPI_Barrier` revealed a severe observer effect: strict synchronization critically disrupts the interconnect pipelining and magnifies OS jitter, effectively degrading overall performance.
- **Scaling Paradigms and Early Stopping:** The weak scaling analysis highlighted a fundamental dichotomy. Scaling the swarm size mathematically forces an artificial speedup rebound via premature early stopping, whereas maintaining a strictly constant local load exposes the true constraints of the network latency.
- **The Curse of Dimensionality:** Without a proportional expansion of the particle population, the exponential growth of the search hyper-volume severely degrades the convergence probability and triggers massive execution delays.

Ultimately, while the DPSO effectively accelerates high-dimensional optimization problems on distributed memory systems, its maximum parallel efficiency remains intrinsically bound by the communication overhead of the dynamic regrouping phase. These findings provide a concrete baseline for evaluating or developing further algorithmic topologies within the broader scope of this study.

Methodological Limitations and Future Work

While the benchmarking campaign provided deep insights into the DPSO architecture, we acknowledge two primary methodological limitations that warrant further investigation:

- **Early Stopping in Weak Scaling:** The current weak scaling analysis is heavily influenced by the early stopping convergence criterion. Scaling the particle population drastically accelerates the discovery of the global minimum, which prematurely halts execution and artificially skews the baseline time. To isolate and measure the raw computational throughput and true network efficiency, future weak scaling tests should strictly disable early stopping, enforcing a fixed number of iterations to guarantee a constant workload per cycle.
- **Statistical Variance:** The experimental benchmarks were executed across 5 independent random seeds. While sufficient for identifying macroscopic scaling trends and network bottlenecks, 5 iterations represent a statistically limited sample size for a highly chaotic and stochastic heuristic like PSO. Future benchmarking suites should increase the sample size to a minimum of 30 independent runs to significantly reducing variance and yielding more robust confidence intervals for the convergence metrics.

6. Introduction to CPSO

Cooperative Particle Swarm Optimization (CPSO) extends standard PSO by partitioning the original D-dimensional search space into smaller subsets of variables, each optimized by a dedicated sub-swarm. While standard PSO evolves a single swarm over the entire decision vector, CPSO distributes the optimization across multiple sub-swarms, each responsible for a distinct portion of the vector.

Each of these sub-swarms communicates with a global *context vector*, which reconstructs a candidate solution by combining the contributions of all the sub-swarms. Consequently, partial updates cannot be evaluated in isolation: each local improvement must be checked in the context of the current partial solutions provided by the other sub-swarms. In this way, CPSO preserves a global optimization target while distributing the search across local components.

Since each sub-swarm operates on its own subset of variables, the optimization can be parallelized at the sub-swarm level. In our parallel implementation, each rank owns one or more sub-swarms, depending on the number of available MPI ranks and on the number K of sub-swarms, computes the corresponding local particle updates, and then participates in the communication required to rebuild a consistent global state.

The solver alternates between *local* particle updates and *global* context maintenance. Each sub-swarm advances only its active coordinates, re-evaluates its personal vector against the current context, and proposes a new local best. The global context is replaced only when the reconstructed full candidate improves the best cooperative configuration found so far.

Algorithm 2 Cooperative PSO

Require: Objective function f, partition $\{\mathcal{D}_s\}_{s=1}^{k}$ of $[D]$, maximum iterations T, reshuffle frequency R.
Ensure: Best cooperative solution $\mathbf{x}^\star$

```
Build a finite initial context vector c
Initialize k sub-swarms and set x* ← c
Let Merge(c, z, D_s) replace in c the coordinates of D_s with z
for t = 1 to T do
    if t > 1 and t mod R = 0 then
        Reshuffle {D_s}_{s=1}^k
    end if
    for s = 1 to k do
        Update all particles of sub-swarm s on D_s
        Evaluate them in the current context c and update their best memories
        q_s ← best particle currently available in sub-swarm s
        ĉ_s ← Merge(c, q_s, D_s)
        if f(ĉ_s) < f(x*) then
            x* ← ĉ_s; c ← x*
        end if
    end for
    Update stopping criteria and stagnation counters
    if stagnation is detected then
        Apply velocity injection and optional memory reset
    end if
    if a termination criterion is satisfied then
        break
    end if
end for
return x*, f(x*)
```

6.1. CPSO Code Flow

The implementation separates the shared algorithmic setup from the actual execution. The serial and parallel versions rely on the same cooperative model, but use completely different approaches to apply the partial updates.

For each test function, the code builds the objective function, creates the stopping criteria, constructs either a `CPSOSerial` or a `CPSOParallel` solver, and then calls `optimize_raw()`. The returned result is a `CpsoRunArtifacts` object, which stores the main metrics, such as the best position, the best fitness, the fitness history, the number of iterations, the stop reason and, for the MPI implementation, the communication times. Most of the preparation is in `CPSOBase`. This class stores the shared parameters, such as the number of sub-swarms, the number of particles per sub-swarm, the reshuffling frequency and the PSO coefficients. Before starting the optimization, it checks that the number of sub-swarms is not larger than the dimension of the problem, because each sub-swarm must own at least one coordinate. Then it creates random generators from a seed, so that the particle dynamics,

topology generation and global operations can be reproduced.
For the domain decomposition, given a problem of dimension D and k sub-swarms, the coordinates are split as equally as possible. Each sub-swarm receives a list of active dimensions, stored in `active_dims`. These indices map the local coordinates of a sub-swarm and the corresponding coordinates in the original global problem. If D is not exactly divisible by k, the first sub-swarms receive one additional coordinate. During the optimization loop, this assignment is not necessarily fixed: every `shuffle_freq` iterations the solver can reshuffle the global coordinates and update each sub-swarm's `active_dims`.
Once the coordinates are assigned, `CPSOBase` builds the local topology of each owned sub-swarm. In the benchmarks used in this paper, the sub-swarms use a scale-free topology. The topology is represented as an adjacency list and is used inside the local PSO update to decide which neighbours can provide the social reference for each particle, so the local update is not a fully global PSO over the sub-swarm, but a topology-aware local PSO step.
Finally, the `ContextVector` is initialized. To ensure a valid starting point, the solver evaluates random global vectors until it finds a finite fitness, defaulting to the domain midpoint if necessary.
Once the setup is completed, the `run_optimization_loop()` function is called by the solver.

6.2. Main Components

The architecture relies on a set of core components:

- `ContextVector`**:** It evaluates a partial particle update by temporarily replacing only the coordinates owned by a specific sub-swarm, computing the full objective function, and then restoring the previous context used for evaluation.
- `SubSwarm`**:** Tracks (best) positions and (best) velocities in its local coordinates. The `active_dims` array maps these indices to the global context vector. Before moving particles, the sub-swarm recalculates its values against the current global context, and then applies the standard PSO velocity update to explore the domain.
- `CPSOBase`**:** It handles common parameters, random generators, dimension partitioning, local topologies, and the initial context vector of both solvers, delegating the actual execution loop to the concrete ones.
- `CPSOSerial` **and** `CPSOParallel`**:** The serial implementation processes all sub-swarms locally and applies partial improvements greedily. The parallel implementation distributes sub-swarm ownership across MPI ranks and handles the distributed protocol required to synchronize and merge partial updates.
- `CpsoRunArtifacts`**:** The standardized output container storing the final solution, fitness history, stop reasons, and MPI communication timings.

6.3. Serial and Parallel Semantics

The serial and parallel versions implement the same cooperative idea, but they differ in how local improvements are evaluated. The real difference between the two solvers is not the local PSO update itself, but the moment in which a local improvement is allowed to modify the shared `ContextVector`.
In the serial solver, all sub-swarms are owned by the same process. After the common setup, each sub-swarm is initialized and its local best is immediately tested against the current context. If the local best improves the common vector, only the coordinates owned by that sub-swarm are copied into the global context.
The same idea is used inside the main loop. At each iteration the solver visits the sub-swarms one after another. For each sub-swarm, the stored fitness values are first recalculated against the latest context, then one PSO step is performed, and finally the particles are evaluated again through the `ContextVector`. If the sub-swarm best is better than the current global fitness, its active coordinates are accepted immediately.
The serial version follows a greedy sequential semantics: a sub-swarm processed later in the same iteration already sees the changes introduced by the sub-swarms processed before it. The drawback is that the order of the sub-swarms has a visible role: the context can change several times inside the same iteration, and each local step is evaluated with the most recent ver-

sion available at that moment.
The parallel solver works differently because several ranks may improve different parts of the solution at the same time. Each MPI rank owns a contiguous range of sub-swarms and updates only those sub-swarms. However, all ranks must keep the same `ContextVector`. For this reason, a rank cannot immediately write its local improvement into the global context. Instead, the parallel loop is divided into synchronized batches: in batch b, each rank updates at most one of its sub-swarms and compares its best local position with the context available at the beginning of the batch. The rank then sends only the deltas of the modified coordinates produced by that sub-swarm.
This avoids sending a complete candidate vector of dimension D. If a sub-swarm controls only a few coordinates, the rank sends only those deltas, padded to `max_swarm_dims` so that `MPI_Allgather` can be used directly.
After all ranks have exchanged their sparse updates, the solver rebuilds a complete candidate context by applying the gathered deltas to the same batch base context. If the fitness of the new candidate is better than the batch base fitness, it becomes the new shared `ContextVector`. The accepted fitness is then broadcast back to all ranks, so every process continues from the new base vector. If the full candidate does not improve the base fitness, the solver does not discard the whole batch immediately; the salvaged and greedy merge steps, described later, try to recover useful partial updates.
The Stopping criteria are also different for the two solvers. In the serial case, the average particle distance is computed over all sub-swarms directly. In the parallel case, each rank contributes only its owned sub-swarms and the final diversity value is reduced across MPI processes. The stopping decision is also synchronized with an `MPI_Allreduce`, so if one rank reaches a stopping condition the whole parallel run terminates consistently. Velocity injection is handled locally on the owned sub-swarms, but it is triggered from the same synchronized stagnation logic.

6.4. Parallelization Details

The MPI version parallelizes the algorithm over sub-swarms: each sub-swarm works on a subset of the total dimensions, so it can be evolved independently for one local PSO step. The main challenge is keeping the global `ContextVector` updated after several ranks have proposed different partial changes at the same time. At the beginning of `CPSOParallel::run_optimization_loop()`, each process reads its `mpi_rank` and the total number of processes `mpi_size`. Then the global list of k sub-swarms is divided into contiguous ranges through the function `compute_all_swarm_ranges`. The distribution is balanced as much as possible: the first ranks receive one extra sub-swarm when k is not divisible by the number of ranks, while ranks beyond the number of available sub-swarms receive an empty interval.
Since the number of owned sub-swarms can differ by one across ranks, the loop is organized in batch slots. The variable `global_max_local_swarms` is computed with an `MPI_Allreduce` over the local number of sub-swarms. For each batch b, rank r works on the global sub-swarm s:

$$s = \texttt{local\_start\_idx}_r + b$$

only if that index is still inside its ownership interval. Otherwise it participates in the collectives with zero deltas and inactive flags.
Before the main loop starts, the initial global context is synchronized and rank 0 broadcast both `init_vec` and `init_fitness` to all other ranks.
The initialization of the sub-swarms is also done in synchronized batches. In each batch, every rank initializes at most one owned sub-swarm. If the local `gbest` improves the current base fitness, the rank does not send a full candidate vector. Instead, it builds a sparse delta with `pack_swarm_delta`. The delta contains only the difference between the sub-swarm best position and the current base context on the active dimensions of that sub-swarm. These deltas are gathered with `MPI_Allgather`, a candidate full vector is reconstructed, then rank 0 evaluates the full objective value, and the accepted fitness is broadcast back.
The same pattern is used during each optimization iteration. After updating the iteration counter and the stopping manager, the solver may reshuffle the dimension assignment. In that

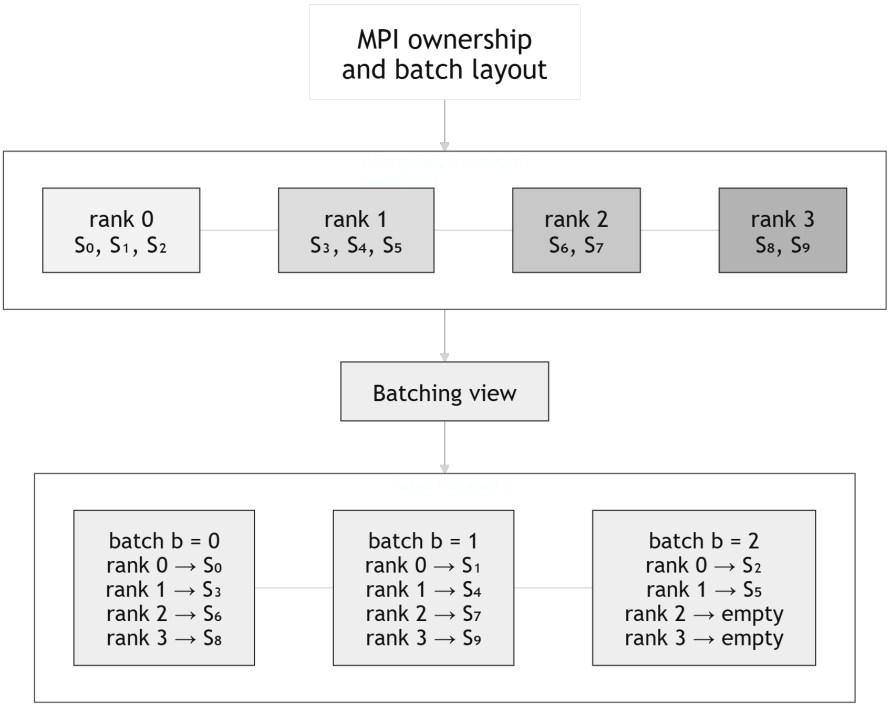

Figure 25: MPI ownership layout used by the parallel CPSO solver. Sub-swarms are assigned to contiguous rank ranges, while the batch view shows which sub-swarm each rank contributes during each synchronized update slot.

case, rank 0 generates one global permutation of the dimensions and broadcasts it. All ranks then call `update_active_dims` with the same new dimensional split. Each rank updates the particle state only for the sub-swarms it owns. For the other sub-swarms, it only keeps the new list of active dimensions, so candidate reconstruction still uses the same indexing on every rank.

The rank saves the current `base_context` and `base_fitness` for its sub-swarms and performs the same local operations like in the serial solvers:

- `recalculate_fitness(...)`: refreshes local memories against the latest accepted global context;
- `update_velocities_and_positions(...)`: performs one PSO iteration;
- `evaluate_and_update(...)`: evaluates the moved particles through the global context and updates personal and sub-swarm bests.

After this local step, the rank prepares two possible messages. The first one is `local_full_delta`, which concern the delta that was applied in the iteration, even if it does not upgrade the context vector. The second one is `local_salvaged_delta`, which is filled only when the local sub-swarm best is better than `base_fitness`. A separate integer flag marks whether this salvaged candidate exists. This distinction is useful because the solver can first try an aggressive merge using all deltas, and if doesn't improve the context vector, can try a more conservative merge using only locally improving deltas (salvaged merge). If a rank has no owned sub-swarm in the current batch, it still participates in the same collectives, but contributes an empty delta and a false salvaged flag.

The solver uses `MPI_Allgather` to collect full deltas, salvaged deltas and salvaged flags from all ranks. It uses `MPI_Bcast` to distribute the fitness values evaluated by rank 0, and also to distribute selected deltas during the greedy fallback. It uses `MPI_Allreduce` for quantities that must become a global decision, such as the maximum number of local sub-swarms, failure propagation, diversity reduction and stopping. A barrier is used around the reshuffle phase, where having all ranks aligned is more important than overlapping work.

The benchmark driver measures the total wall time around the whole solver run, while the helper `time_mpi_call` wraps the MPI

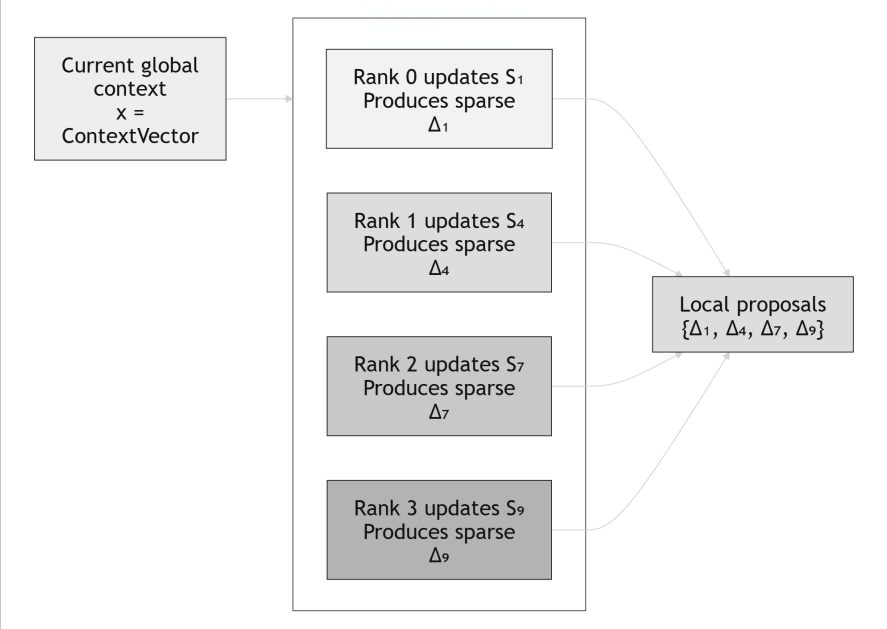

Figure 26: Local proposal phase of one parallel CPSO batch. Each MPI rank updates one owned sub-swarm and produces a sparse delta over the corresponding active dimensions.

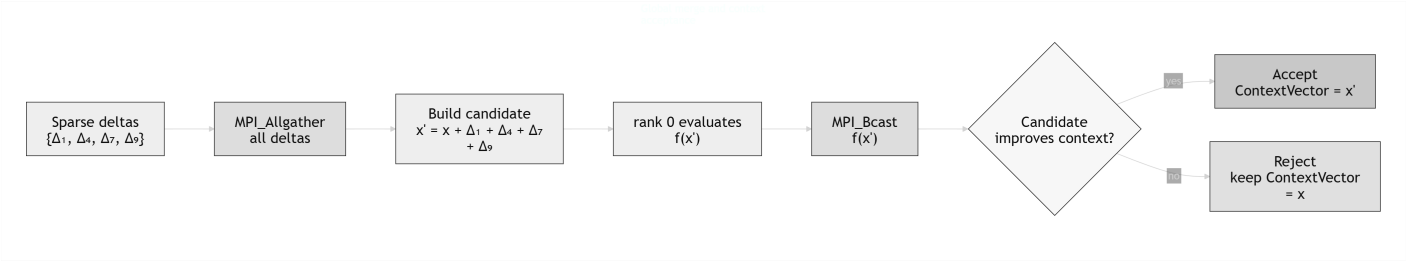

Figure 27: Global merge and context acceptance phase of one parallel CPSO batch. The sparse deltas are gathered, combined into a candidate context, evaluated, broadcast and then either accepted or rejected.

primitives and accumulates their time in `MpiTimingStats`. At the end of the run, `fill_parallel_timing_artifacts` stores the time spent in `Allgather`, `Bcast`, `Allreduce`, barriers, waiting and estimated computation. These values are reduced with a maximum across ranks, because the slowest rank is the one that determines the observed parallel runtime.
Finally, numerical failures are synchronized instead of being left as local exceptions. When a rank detects a problem, `sync_failure_state` first uses an `MPI_Allreduce` to determine whether any process failed, then selects the failing rank and broadcasts the message. This avoids a common MPI problem where one process exits while the others are still waiting inside a collective.

Overall, the parallelization is not just a matter of distributing particles. It is a distributed protocol for proposing, exchanging and validating partial changes to a shared cooperative context. The next subsection focuses on the merge policy itself: full merge, salvaged merge and greedy fallback.

6.5. Parallel Merge Policy

At the end of a batch, several ranks may have produced local proposals at the same time. Each proposal is good with respect to the context used by that rank during its local update, but this does not automatically mean that all proposals are good when applied together. Two partial updates can interact badly, because the objective function is still evaluated on the complete vec-

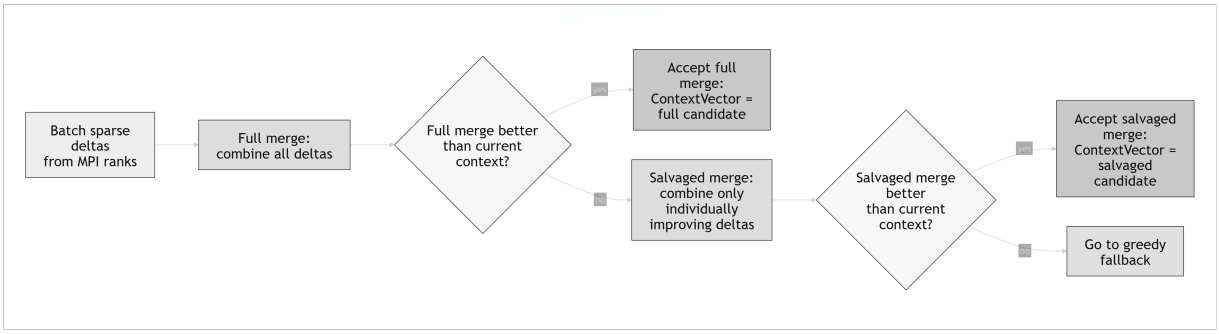

Figure 28: Full and Salvaged merge used in the optimization loop.

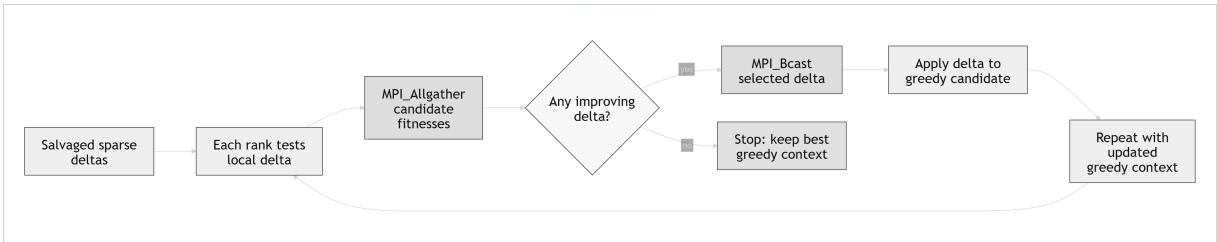

Figure 29: Greedy fallback used when both the full and salvaged merge fail.

tor. For this reason, the solver uses a sequence of merge attempts, from the most aggressive to the most conservative one.

The first attempt is the *full merge*. The solver applies all `local_full_delta` messages collected in the batch to the same `base_context`. This produces a complete candidate vector containing the best local proposal from every participating rank. Rank 0 evaluates this full candidate with the original objective function and, if its fitness is better than `base_fitness`, the new vector is accepted and all ranks update their `ContextVector`. This is the ideal case, because all parallel local work contributes to the new global context.

If the full merge does not improve the objective, the solver tries a *salvaged merge*. In this case it uses only the deltas marked by `local_has_salvaged_candidate`. These are the proposals that improved the base context when considered individually. This second attempt is less aggressive: it discards updates that may be useful later, but that are not strong enough to justify changing the current context in this batch. Again, the result is evaluated as a complete vector before being accepted.

The last fallback is the *greedy merge*. Instead of applying all salvaged deltas together, the solver tests only the proposals that were already individually improving with respect to the batch `base_context`. At each greedy round, every rank evaluates whether its own salvaged delta improves the current greedy context. The candidate fitness values are collected with `MPI_Allgather`; all ranks can then identify the best improving delta, and that delta is broadcast with `MPI_Bcast`. Once a delta is accepted, it is removed from the local candidate set and the procedure repeats. If no candidate improves the current greedy context, the loop stops.

6.6. Benchmark Methodology

The scope of benchmarks is to evaluate the optimization behavior and the cost of the parallel communications. All runs use the benchmark factory defined in `CPSOBenchmarkUtils.hpp`, so the serial and parallel drivers execute the same set of test functions through the same `TestFunction` interface. The result of each solver run is stored in a `CpsoRunArtifacts` object and then converted to the common benchmark output format. Unless stated otherwise, the plotted values are averages over the selected seeds and over the benchmark functions, not only based on a single run measurements.

The main configuration uses a scale-free local topology inside each sub-swarm, with some fixed hyperparameters (`shuffle_freq = 50`, `stagnation_patience = 50`, inertia de-

creasing from 0.9 to 0.4, and acceleration coefficients $c_1 = c_2 = 1.49618$). The strong-scaling experiments use $k = 32$ sub-swarms and 8 particles per sub-swarm, with dimensions 32, 64 and 128. The process counts are `1`, `2`, `4`, `8`, `16` and `28`. Dimensions 32 and 64 use `max_iters = 10000`, while dimension 128 uses `max_iters = 20000`. The main seeds are `123`, `456`, `789`, `2024` and `4242`, so the reported curves are not based on a single lucky run.

The `stagnation_patience` controls the period after which a stagnant sub-swarm receives velocity injection. The global stopping manager uses a separate maximum stagnation window, set to $\max(100, \texttt{max\_iters}/4)$ in the benchmark drivers.

The final benchmark discussion is organized around three questions: how much the parallel version accelerates a fixed problem, how much of the runtime is spent in communication, and how sensitive the results are to the relation between the number of sub-swarms k and the number of MPI ranks.

6.7. Benchmark Results

The strong scaling results show that the parallel implementation gives a clear runtime advantage, compared with the serial one. With the baseline configuration $k = 32$, the best point is often around `np = 16`. For example, with dimension 64 the speedup against the serial version is about 7.82 at `np = 16`, while it decreases to about 6.81 at `np = 28`. With dimension 128, the pattern is similar, but the larger problem amortizes the parallel overhead better: the speedup is about 10.83 at `np = 16` and about 9.24 at `np = 28`.

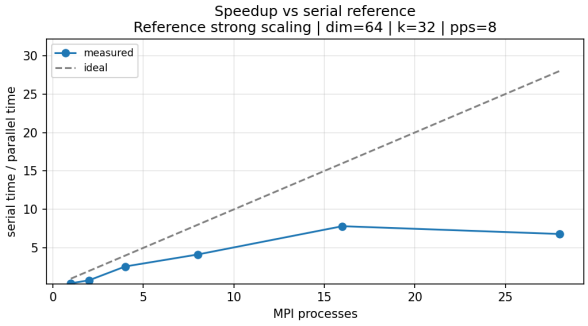

Figure 30: Strong-scaling speedup at dimension 64, using the baseline configuration with $k = 32$ sub-swarms.

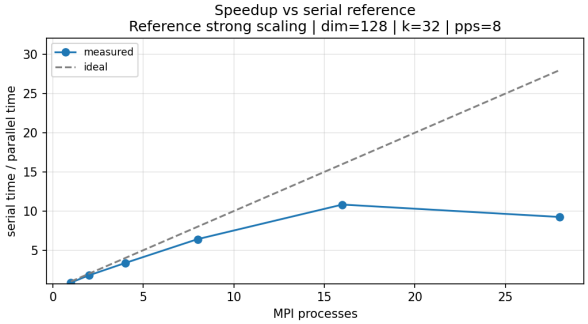

Figure 31: Strong-scaling speedup at dimension 128, using the baseline configuration with $k = 32$ sub-swarms.

The communication plot explains why the largest process count is not always the best one. Increasing the number of ranks reduces the amount of local PSO work per rank, but it also increases the synchronization needed to keep the context vector consistent. In the $k = 32$ baseline, `np = 28` has more parallel resources than `np = 16`, but the additional communication can dominate the benefit of the extra ranks: CPSO parallelizes the local search, but the global context still has to be merged and validated.

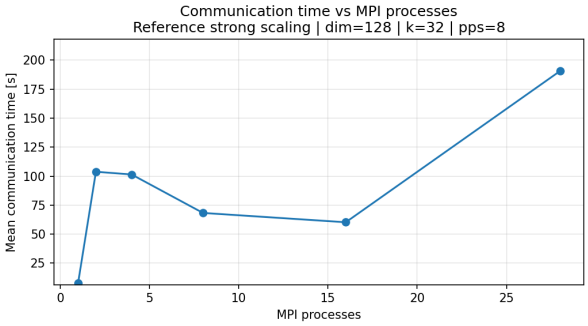

Figure 32: Total MPI communication time in the strong-scaling suite at dimension 128.

The weak-scaling experiment increases both the global dimension and the number of sub-swarms with the number of MPI ranks, using $D = 8n_p$ and $k = n_p$, while keeping the total number of particles fixed at 224. This configuration studies a weak-scaling regime under a fixed global particle budget: as the number of MPI ranks increases, the cooperative decomposition becomes finer and the particles are redistributed across a larger number of sub-swarms.

The purpose of this experiment is to evaluate whether the MPI implementation remains effec-

tive as the global search space grows and the cooperative structure becomes increasingly distributed. The parallel-only wall-time plot is therefore the clearest view of this behavior: it avoids compressing the parallel curves under a much larger serial runtime and shows directly how the MPI version behaves as the dimension and the number of cooperative components increase.

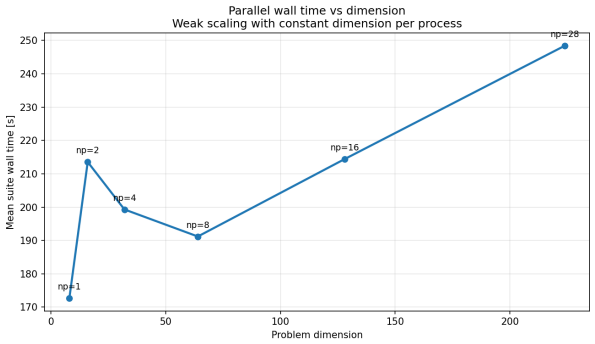

Figure 33: Weak-scaling wall time of the parallel CPSO solver under a fixed global particle budget, with $D = 8n_p$, $k = n_p$, and a total particle count of 224, where n_p is the number of MPI ranks.

An additional study is the configuration with $k = 28$. In this suite the largest process count satisfies `k = np = 28`, so each active rank owns exactly one sub-swarm. The two cases use the same serial and parallel workload within each comparison, but they are not identical to the main $k = 32$ baseline: the dimension 64 case uses 16 particles per sub-swarm, while the dimension 128 case uses 8 particles per sub-swarm. Under these workloads, the speedup is about 15.68 for dimension 64 and about 16.26 for dimension 128. This suggests that the relationship between k and the MPI rank count is structurally important.
An important detail is that these favourable $k = 28$ cases do not depend on the dimension being perfectly divisible by k. In fact, $64 \bmod 28 = 8$ and $128 \bmod 28 = 16$. The implementation handles this by assigning one additional coordinate to the first sub-swarms. Therefore, the strongest interpretation is not that perfect divisibility is required, but that a clean ownership mapping between sub-swarms and MPI ranks can reduce imbalance and improve the ratio between useful local work and synchronization overhead.

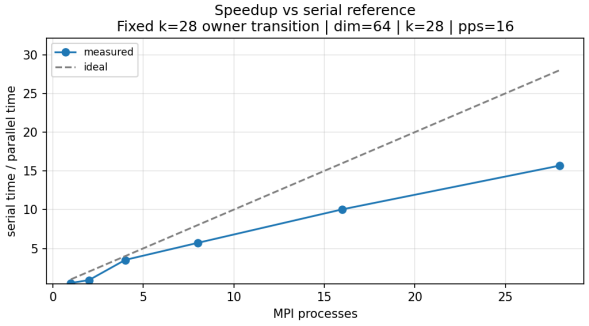

Figure 34: Sensitivity study with $k = 28$ sub-swarms and 16 particles per sub-swarm at dimension 64.

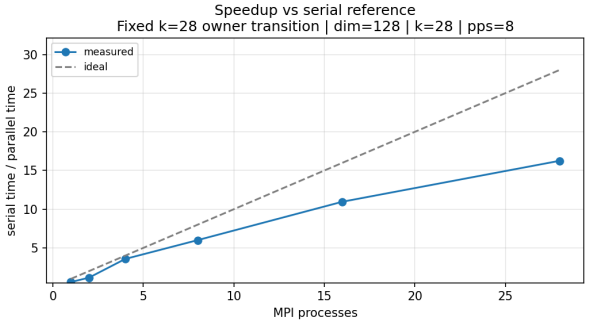

Figure 35: Sensitivity study with $k = 28$ sub-swarms and 8 particles per sub-swarm at dimension 128.

6.8. Final CPSO Discussion

The CPSO implementation highlights a fundamental trade-off in distributed optimization: while decomposing the search space into sub-swarms provides a naturally parallelizable workload, the algorithm's strict dependency on a shared global context mandates a rigorous synchronization protocol. The sub-swarms cannot evolve in true isolation; every local trajectory must be continuously validated against the global `ContextVector`.
From an architectural perspective, this work proposes a distributed pipeline that connects local PSO updates with global state validation. Instead of replicating the full state across processes, the implementation uses sparse updates and a structured merge strategy. This reduces communication costs while preserving the cooperative behaviour of the algorithm and enabling a detailed analysis of MPI overhead.